provadis Hochschule

Bachelorarbeit

OSS Risk Radar: Risk Assessment for Open Source Dependencies

zur Erlangung des akademischen Grades
'Bachelor of Science' B.Sc.
im Studiengang 'Informatik'

vorgelegt dem Fachbereich Informatik und Wirtschaftsinformatik der
Provadis School of International Management and Technology
von

Eric Krause
Matrikelnummer: D720
BIN-T23-F-4
eric.krause@telekom.de

Erstgutachter:	Prof. Dr. Lamya Abdullah
Zweitgutachter:	[Second Reviewer]
Ende der Bearbeitungsfrist:	[Date]

Abstract

Modern software depends heavily on open-source components, yet downstream projects remain exposed when an upstream repository quietly stops being maintained. This thesis asks to what extent publicly observable repository health and maintenance indicators can predict whether an open-source dependency becomes inactive within a twelve-month horizon, and how the resulting risk score can support risk-based dependency triage. Following a Design Science Research approach, the work develops and evaluates OSS Risk Radar, an artifact that constructs a leakage-controlled historical dataset from GH Archive event data and public package metadata, engineers observation-time activity, responsiveness, and contributor-concentration features, and trains calibrated inactivity-risk models.

On a repository foundation dataset of 5,000 projects and 50,000 quarterly observation snapshots, a gradient-boosted model reaches a held-out ROC-AUC of 0.945 with a Brier score of 0.089, clearly outperforming a trivial recency baseline (ROC-AUC 0.655). The skill largely persists under a repository-disjoint split (0.940) and among repositories that are still active at the observation time (0.883), indicating that the model detects early decline rather than merely restating existing quietness. A human-actor filter is validated by ablation, and security-practice signals are kept as parallel triage context rather than as predictors of inactivity. A practical demonstrator applies the deployed model per repository to the dependency repositories of a real project, producing a calibrated, confidence-aware, and evidence-backed ranked triage view. The score is positioned as a decision-support signal for prioritizing dependency review, not as a verdict on abandonment.

Declaration of Authorship

I declare that I wrote this thesis independently and used only the sources and tools listed in the bibliography and acknowledgements.

Place, date	Signature

Contents

List of Figures

List of Tables

List of Abbreviations

API Application Programming Interface

CVSS Common Vulnerability Scoring System

ML Machine Learning

OSS Open Source Software

1 Introduction

1.1 Motivation and Problem Statement

Modern software systems are not developed in isolation, but are commonly assembled from first-party code, third-party components, and open-source software dependencies. In this thesis, open-source software is understood as software distributed under a license that complies with the Open Source Definition, which includes rights such as use, modification, and redistribution [1]. From a secure software development perspective, such dependencies are relevant because the NIST Secure Software Development Framework explicitly includes the acquisition and maintenance of well-secured third-party and open-source components as part of secure development practice [2]. Similarly, the SLSA framework addresses software supply-chain integrity by focusing on artifacts, build processes, provenance, and related security controls [3].

The practical relevance of dependency management becomes visible in both empirical studies and real-world incidents. Miller et al. studied open-source dependency abandonment in the npm ecosystem and found that abandonment is common among widely used packages while downstream projects often remain exposed, particularly through indirect dependencies that developers have limited ability to react to [4]; in an earlier interview study they found that affected developers often lacked clear strategies and felt they were improvising [5]. Delayed maintenance also prolongs security exposure: Prana et al. analyzed dependencies in 450 Java, Python, and Ruby projects and found that many vulnerabilities persisted throughout the observation period, taking several months to resolve on average [6].

Real-world incidents illustrate the potential impact. The 2016 unpublishing of the small npm package `left-pad` disrupted many directly and transitively dependent projects [7]; the 2017 Equifax breach exploited an unpatched Apache Struts vulnerability and led to a settlement of up to $700 million [8], [9]; and the XZ Utils backdoor showed that maintainer trust itself can become part of the attack surface through social-engineering takeover [10].

These examples do not imply that every inactive repository is insecure, abandoned, or harmful. A project may be stable, temporarily paused, or maintained through channels that are not fully visible in public repository data. However, they show that dependency health, maintainability, and responsiveness are relevant for secure software supply chains. Therefore, this thesis treats repository inactivity not as a definitive indicator of project death, but as an operational proxy for elevated maintenance risk. The central question is whether publicly observable repository signals can help estimate this risk early enough to support dependency triage and monitoring.

1.2 Research Question and Objectives

The central research question of this thesis is:

> To what extent can a machine-learning-based scoring tool, using only publicly observable repository health and maintenance indicators, predict 12-month OSS dependency inactivity, and how can this inactivity-risk score be combined with parallel security-practice signals to support risk-based dependency triage in secure dependency management?

To answer this research question, the thesis pursues the following objectives:

1. Define an operational inactivity label for open-source repositories within a 12-month prediction horizon.
2. Construct a reproducible dataset from publicly observable repository metadata and dependency-related sources.
3. Engineer observation-time activity and maintenance features for the inactivity model, and surface selected security-practice signals as parallel triage context rather than as inputs to the inactivity prediction.
4. Train and compare simple baselines and machine-learning models for inactivity-risk prediction.
5. Evaluate the models using discrimination and calibration metrics that are suitable for risk scoring.
6. Demonstrate how the trained model can be applied per repository to a set of dependency repositories to support risk-based triage.

The research follows a Design Science Research-oriented approach because the thesis develops and evaluates an artifact: an inactivity-risk scoring workflow for open-source

dependencies [11], [12]. The empirical part evaluates whether the artifact produces useful predictive signals, while the practical demonstrator shows how the scoring workflow can be integrated into dependency monitoring.

1.3 Scope and Assumptions

The scope of this thesis is limited to open-source software dependencies whose source repositories and package metadata are publicly observable. The empirical dataset focuses on repositories that can be associated with an open-source license. Where GitHub license metadata is used, matched license keys and names are treated according to GitHub's SPDX-based license information [13]. The thesis does not investigate private repositories, closed-source vendor components, or dependencies whose source repository cannot be mapped with sufficient confidence.

The prediction target is repository inactivity within the interval $[t, t + 12 \text{ months}]$. Inactivity is operationalized primarily through the absence of commits and releases or tags during the prediction horizon. This definition follows the idea that revision activity can be used as an observable signal for survival-style analysis of open-source projects [14]. Issue and pull-request activity are treated as predictor variables instead of label components, because they can indicate maintenance interaction without necessarily changing the released software artifact. Responsiveness-related features are interpreted cautiously, since bot-generated responses can distort measurements such as time to first response in pull-request workflows [15].

The thesis assumes that publicly observable repository signals can provide useful but incomplete evidence about future inactivity. It does not assume that these signals fully explain project sustainability. This distinction is important because previous research shows that cross-project health indicators can have different meanings depending on project context [16]. As a result, the model output is interpreted as a triage aid rather than as an automated decision system.

1.4 Contributions

Realizing the objectives above, this thesis contributes an operational 12-month inactivity-risk label and a reproducible construction workflow for it; an empirical evaluation of whether public repository health and maintenance indicators support machine-learning-based inactivity prediction, benchmarked against simple baselines on both discrimination

and probability calibration, with security-practice signals kept as complementary triage context rather than as inactivity predictors; and a per-repository demonstrator that turns the score into a ranked triage view. These contributions are intended to support software teams prioritizing dependency review effort. The goal is not to replace expert judgement, but to provide a measurable and reproducible signal that helps identify dependencies deserving closer attention.

1.5 Structure of the Thesis

The remainder of this thesis is structured as follows. Chapter 2 introduces the theoretical background, including software supply-chain security, open-source sustainability, project health metrics, and machine-learning basics for tabular risk scoring. Chapter 3 discusses related work on inactivity and abandonment operationalizations, repository health metrics, and security-practice indicators, and positions the artifact relative to them. Chapter 4 presents the research design, inactivity label, dataset construction, feature engineering approach, models, baselines, and evaluation strategy. Chapter 5 describes the technical implementation of the data pipeline and scoring workflow. Chapter 6 reports and discusses the empirical results, including predictive performance, calibration, feature importance, and error analysis. Chapter 7 demonstrates the trained model applied per repository to a set of dependency repositories as a ranked triage view. Chapter 8 consolidates the threats to validity and their mitigations. Chapter 9 concludes the thesis and outlines limitations and opportunities for future work.

2 Background

This chapter introduces the conceptual foundations required for the thesis. It first explains the software supply-chain security context in which open-source dependencies are used and assessed. It then discusses open-source software sustainability and repository health metrics. Afterwards, it outlines important limitations of metric-based project assessment. Finally, it introduces the machine-learning concepts that are relevant for tabular inactivity-risk scoring.

2.1 Software Supply-Chain Security

Software supply-chain security focuses on reducing risks that arise when software artifacts, build processes, dependencies, and distribution channels are not sufficiently controlled or verifiable. Two frameworks establish why dependencies fall within this scope: the NIST Secure Software Development Framework treats the acquisition and maintenance of well-secured software components as part of secure development practice [2], and the SLSA framework defines security-oriented requirements for artifacts and build processes, including provenance and integrity controls [3]. Together they motivate monitoring dependencies beyond simple version updates.

Open-source dependencies are especially relevant because they are reused across many downstream projects. Open-source software is defined by licensing conditions that allow use, modification, and redistribution under criteria specified by the Open Source Initiative [1]; these freedoms enable broad reuse but also mean that downstream users depend on external maintainers and public infrastructure, so the maintenance activity of an upstream repository becomes relevant for downstream risk assessment. Supply-chain security thus provides the practical context for inactivity-risk scoring: the goal is not to determine whether a dependency is secure in general, but to estimate whether publicly observable repository signals indicate an elevated risk of future inactivity, supporting monitoring, replacement planning, or prioritization of manual review.

This context also explains why the unit of analysis is a single repository. Enumerating

which repositories a project depends on is the task of established software-composition-analysis tooling, such as the OSS Review Toolkit [17]; this thesis therefore treats the dependency inventory as an external input and focuses on scoring each individual repository. In the practical demonstrator (Chapter 7), the inactivity-risk model is applied per repository to a set of dependency repositories to produce a ranked triage view. Supply-chain security is therefore the reason to look beyond version numbers and consider the maintenance trajectory of the upstream repositories that a project depends on.

2.2 Open-Source Sustainability and Project Health

Open-source sustainability describes whether a project can continue to provide value and maintain its software over time. Project health is commonly assessed through observable indicators such as activity, responsiveness, contributor participation, and release behavior. The CHAOSS project provides standardized metrics and metrics models for understanding open-source community and project health [18]. Its Starter Project Health Metrics Model is intended as an entry point for measuring selected aspects of project health in a structured way [18].

Repository activity is one category of observable project health. Examples include commits, releases, tags, issues, pull requests, and contributor activity. These signals are useful because they can often be collected from public repository platforms and package metadata sources. Prior research has used revision activity and project-level attributes to study open-source project survival and inactivity-related outcomes [14]. In this thesis, such activity signals form the basis for predicting whether a dependency repository becomes inactive within a 12-month horizon.

Security-practice indicators form a second category of observable signals. OpenSSF Scorecard is an automated tool that assesses open-source projects using security-related checks [19]. These checks do not directly measure future maintenance activity, but they may provide additional information about project practices, repository configuration, and security posture. As explained in Section 4.1 and Section 3.3, this thesis deliberately keeps such security-practice signals as a *parallel* triage context rather than feeding them into the inactivity model, so that no claim is made that they improve inactivity prediction.

However, project health is broader than any single metric. Goggins et al. argue that making open-source project health transparent requires structured measurement and interpretation rather than isolated indicators [20]. This is important for the thesis

because inactivity-risk scoring should not be interpreted as a complete judgement of project sustainability. Instead, the score is treated as a limited operational signal for dependency triage.

These categories of observable health indicators — commit activity, contributor activity, issue and pull-request activity, release activity, age and popularity, and security practice — motivate the feature groups introduced later; their example signals, public sources, and expected relevance to inactivity risk are summarized in Appendix A.1 (Table A.1), and the exact per-feature definitions of the implemented set are given in Section 4.7 and Appendix A.2. Security-practice indicators are included for completeness but, as discussed in Section 3.3, are kept as parallel triage context rather than as inputs to the inactivity model.

2.3 Metric Pitfalls and Limitations

Although repository metrics are useful, they can be misleading when interpreted without context. Yehudi et al. show that individual context-free online community health indicators are insufficient for identifying open-source software sustainability across heterogeneous projects [16]. This means that a metric value can have different meanings depending on the project type, maturity, governance model, and maintenance style.

One example is commit activity. A low number of recent commits may indicate reduced maintenance, but it may also indicate that a project is stable and does not require frequent changes. Similarly, a high number of issues may indicate an active user community, but it may also indicate unresolved maintenance problems. Therefore, this thesis does not treat single metrics as direct proof of project health or project failure. Instead, multiple signals are combined and evaluated empirically.

Responsiveness metrics also require caution. For example, time-to-first-response can be affected by automated bot responses in pull-request workflows. Hasan et al. show that bot-generated first responses can influence the interpretation of responsiveness measurements in GitHub pull requests [15]. For this reason, responsiveness-related indicators should be interpreted carefully and, where possible, separated into human and automated activity.

A further limitation is that public repository data may be incomplete. Development may move to another platform, releases may be published outside the observed repository, or maintainers may communicate through channels that are not captured by

GitHub metadata. Package-to-repository mappings may also be ambiguous or missing. Therefore, the empirical results of this thesis must be interpreted as evidence based on publicly observable data, not as a complete representation of all project activity.

These limitations influence the design of the inactivity label. Inactivity is not defined as project death or abandonment. Instead, it is operationalized as the absence of selected observable activity signals within a defined prediction horizon. This makes the label measurable and reproducible, while still acknowledging that it is only a proxy for elevated maintenance risk.

2.4 Machine Learning Basics for Tabular Risk Scoring

The prediction task in this thesis is binary classification: at observation time t a repository is represented by features derived from public metadata and activity signals, and the model estimates the probability that it becomes inactive in $[t, t + 12 \text{ months}]$. Because inactivity is by definition closely related to recent activity, a recency-based baseline (time since the last observed commit or release) is required as a reference point; a machine-learning model is only worth reporting if it improves over it. Among learned models, logistic regression serves as an interpretable probabilistic baseline, while tree-based models such as gradient boosting capture non-linear feature interactions [21], [22], [23], which suits tabular repository metadata where activity and maintenance indicators may interact non-linearly.

Model evaluation should consider both discrimination — how well a model separates inactive from active repositories, measured by the area under the ROC or precision-recall curve, the latter being relevant under class imbalance [24] — and calibration, whether predicted probabilities match observed outcome frequencies, which matters because a risk score should be interpretable as a probability-like signal rather than only a ranking [25]. Finally, the dataset should be split in a time-aware rather than random way, so that information from later periods cannot influence model development for earlier ones; this matches the intended use of historical information to predict future inactivity.

The following chapter discusses related work on open-source inactivity, repository health measurement, and security-practice indicators in more detail.

3 Related Work

This chapter positions the thesis relative to existing work. Chapter 2 introduced the general concepts of supply-chain security, project health metrics, and tabular machine learning. This chapter goes one step further and discusses concrete prior approaches, their operationalizations, and the gap that this thesis addresses. The aim is comparative rather than introductory: for each strand of work, the relevant difference to the present inactivity-risk artifact is made explicit.

3.1 OSS Inactivity and Abandonment Operationalizations

A first strand of work studies how open-source inactivity or abandonment can be defined and measured. Robinson et al. apply survival-style analysis to open-source Python projects and use revision activity as an observable signal for project survival [14]. This supports the idea that activity-based signals can be used to reason about future project state, but survival analysis typically estimates time-to-event over a population rather than producing a per-repository forward-looking risk score at a fixed horizon.

Miller et al. study dependency abandonment in the npm ecosystem and show that abandonment is common among widely used packages and that downstream projects often remain exposed, including through transitive dependencies [4]. Their earlier interview study shows that developers frequently lack clear strategies for responding to abandonment [5]. These works motivate the practical need for an early-warning triage signal, but they characterize abandonment retrospectively and at the ecosystem level rather than predicting a 12-month inactivity outcome for an individual repository from point-in-time features.

The present thesis differs in three ways. First, it fixes a 12-month prediction horizon and a snapshot-based observation date, so that prediction is strictly forward-looking. Second, it derives the label from a multi-signal maintenance rule rather than from a single revision-activity threshold. Third, it treats issue and pull-request activity as

predictors rather than as label components, to avoid conflating maintenance interaction with the released-artifact signal.

3.2 Repository Health Metrics and Dashboards

A second strand concerns standardized project health measurement. The CHAOSS project provides metrics models, including the Starter Project Health Metrics Model, as structured entry points for measuring project health [18]. Goggins et al. argue that making project health transparent requires structured measurement and interpretation rather than isolated indicators [20]. These efforts provide a vocabulary of observable signals that informs the feature engineering of this thesis.

However, dashboards and metrics models are primarily descriptive: they surface indicators for human interpretation rather than predicting a future outcome. Yehudi et al. show that individual context-free health indicators are insufficient for identifying sustainability across heterogeneous projects [16]. This thesis takes that critique as a design constraint: instead of presenting single indicators, it combines many observation-time signals into a calibrated predictive model and evaluates the combination empirically, while explicitly acknowledging that the resulting score is a limited operational proxy.

3.3 Security-Practice Indicators

A third strand concerns security-practice signals. OpenSSF Scorecard provides automated security-related checks for open-source repositories [19], and supply-chain frameworks such as the NIST SSDF and SLSA motivate why dependency security posture matters [2], [3]. These checks describe present-day security practices rather than future maintenance activity.

This distinction is important for the present work. As stated in Section 4.1, OpenSSF Scorecard is used as a parallel security-posture signal in OSS Risk Radar and is deliberately not part of the inactivity model feature vector. The thesis therefore does not claim that security-practice checks improve inactivity prediction; it keeps the predictive target focused on activity and maintenance signals, while presenting security posture as complementary context for triage.

3.4 Responsiveness Measurement Caveats

Several features in this thesis describe responsiveness, such as issue first-response time and pull-request response and merge latency. Hasan et al. show that bot-generated first responses can distort time-to-first-response measurements in GitHub pull requests [15]. This thesis treats responsiveness features as reconstructed proxies derived from GH Archive event state, interprets them cautiously, and does not treat them as canonical CHAOSS metrics, in line with the caveats discussed in Section 2.3.

3.5 Research Gap

Prior work provides operationalizations of inactivity and abandonment, standardized health metrics, and security-practice indicators, but it largely characterizes project state descriptively or retrospectively. The gap addressed by this thesis is a reproducible, forward-looking, calibrated per-repository inactivity-risk model with a fixed 12-month horizon, a multi-signal maintenance label, leakage-controlled point-in-time features, and an explicit separation between the inactivity prediction target and parallel security-posture signals.

4 Methodology

This chapter describes the methodological design used to answer the research question. The thesis follows a Design Science Research-oriented approach, because the work develops and evaluates an artifact for a practical software engineering problem: an inactivity-risk scoring workflow for open-source dependencies [11], [12]. The artifact is implemented as OSS Risk Radar, a decision-support tool for open-source dependency maintenance and software supply-chain risk triage [26].

The methodology combines artifact construction with historical supervised learning. Instead of using manually assigned or synthetic labels, the training dataset is built from real open-source repositories and historical GitHub event data. For each repository, the dataset builder creates observation snapshots at defined points in time. Each snapshot contains only features that would have been available at or before the observation date t. The label is derived from the following 12-month period $(t, t + 12 \text{ months}]$. This design reflects the intended use case: estimating future inactivity risk from observable signals, rather than explaining inactivity after it has already occurred.

4.1 Research Design

The research design consists of three connected parts. First, OSS Risk Radar is developed as a software artifact that can ingest dependency or repository information, enrich it with public signals, and produce inactivity-oriented risk scores. Second, a historical supervised training dataset is constructed from real open-source repositories. Third, trained model artifacts are evaluated quantitatively and qualitatively to determine whether the scores are useful for risk-based dependency triage.

The artifact is not designed as a vulnerability scanner or as a definitive trust score. It does not claim that a repository is secure or insecure. Instead, it estimates whether observable maintenance and activity signals indicate elevated inactivity risk. This distinction is important because inactivity is only one aspect of dependency risk. A repository can be inactive without being vulnerable, and a repository can be active

while still having security weaknesses. Therefore, the output of the inactivity model is interpreted as a decision-support signal for prioritization and manual review.

The scope of the predictive model must be stated precisely, because the research question also refers to security-practice signals. OpenSSF Scorecard is used as a *separate* security-posture signal in the surrounding enrichment workflow [19], *not* as part of the predictive feature vector: the inactivity classifier is trained exclusively on activity, maintenance, responsiveness, and concentration signals, and security-practice indicators are presented to the analyst as a parallel posture output rather than as predictors of inactivity. This separation avoids overclaiming that security-practice checks improve inactivity prediction and keeps the predictive target conceptually clean.

4.2 Iterative Artifact Development

OSS Risk Radar was developed iteratively. The first iteration validated the end-to-end workflow with a small pilot training run. This run was used to check whether the pipeline could create historical snapshots, train a model, export an artifact, and return risk scores for manually inspected repositories. It was not treated as the empirical basis of the thesis and is not used in any reported result.

Manual inspection of the first model outputs revealed that the initial scoring behavior was not sufficiently plausible in all cases. A small number of repositories were used as informal, formative review examples during this diagnostic step — one operationally active project scored implausibly high, and others scored low where a higher or more nuanced assessment seemed warranted — and are reported as qualitative development evidence rather than as a measure of model accuracy. These cases were used as an error-analysis step, after which the feature set and training setup were revised to better separate activity, release behavior, contributor dynamics, responsiveness, backlog pressure, and concentration-related risk. This process is consistent with the Design Science Research framing of the thesis: the artifact was built, evaluated, diagnosed, and refined based on observed shortcomings.

The second iteration produced the dataset and model artifacts used for the reported evaluation. It expanded the empirical basis to the repository-first foundation dataset described in Section 4.4, whose seed strata are used only as sampling provenance while the supervised label is derived from future repository behavior.

4.3 Definitions and Prediction Target

The central unit of analysis is a GitHub repository. In the training dataset, a row represents a repository observation snapshot at a specific observation date t. The prediction task is binary classification:

> Given the observable repository and package state at time t, predict whether the repository becomes inactive within the following 12 months.

The exported training label is named `label_inactive_12m`. Internally, the dataset builder first computes a maintenance variable named `maintained_12m`. The inactivity label is then derived as the inverse of this maintenance variable. If the future window is incomplete, the row remains unlabeled and is not used for supervised model fitting.

The future label window is defined as:

$$(t, t + 12 \text{ months}]$$

The current implementation treats a repository as maintained within the 12-month horizon if it is not archived or deleted by $t + 12$ months and if at least two of the following future-window activity checks are fulfilled:

1. human commits occur in at least three distinct months,
2. at least two contributors are active,
3. at least one release or package version publication occurs,
4. at least two pull requests are merged.

This definition deliberately avoids treating a single event as sufficient evidence of maintenance. At the same time, it remains operationally measurable from historical public data. The merged-pull-request condition is used as a practical proxy for broader maintainer interaction, because GH Archive data does not reliably identify all maintainer responses without additional issue-comment attribution logic [27].

The specific thresholds in this rule (the two-of-four requirement and the numeric cut-offs for months, contributors, releases, and merged pull requests) are heuristic design choices rather than externally validated constants. They were chosen to require sustained rather than incidental activity. Because the label definition directly influences class prevalence, the sensitivity of the results to these thresholds is treated as a robustness concern and discussed in Section 4.12.

4.4 Data Sources

The dataset construction uses public and reproducible data sources wherever possible. GH Archive is used as the primary source for historical GitHub event data. GH Archive provides public GitHub event streams as hourly archives [28]. The dataset builder parses these archives into normalized repository histories, including commits, issues, pull requests, releases, stars, and forks.

Repository and package metadata are enriched through multiple adapters. deps.dev is used for package and dependency metadata where package-to-repository mapping is required [29]. GitHub metadata is used for stable repository fields such as creation time, default branch, and fork state. OpenSSF Scorecard is used by the broader OSS Risk Radar enrichment workflow as a separate security-practice signal source [19], but, as stated in Section 4.1, it is not part of the historical inactivity feature vector. Present-day GitHub activity totals are not used as historical observation-time features, because they would leak information from after the observation date into older snapshots.

The repository-first foundation dataset is drawn from a generated seed of public, non-fork GitHub repositories that have a license object and meet a minimum star threshold (default 100 stars), stratified into active, dormant, and archived buckets that serve only as sampling provenance — the supervised labels come exclusively from future activity in the 12-month window [27]. The seed deliberately oversamples inactive repositories (before sampling, the documented default frame is roughly 45% active versus 55% dormant or archived), which raises the number of positive examples available for supervised learning but also lifts the inactive base rate above the natural OSS rate: in the held-out test slice it is approximately 0.56, close to balanced. Predicted probabilities are therefore calibrated to this sampled prevalence rather than to a specific deployment population, a point revisited for calibration and precision in Section 4.11 and Section 4.12.

4.5 Dataset Construction Workflow

The historical dataset builder creates an inspectable dataset through a sequence of intermediate steps: it loads repository candidates from the foundation seed, resolves repository identities, enriches stable metadata through GitHub, parses GH Archive event files into normalized repository histories, builds quarterly observation snapshots, computes observation-time features using only data available at or before t, computes 12-month labels using only the future window $(t, t + 12 \text{ months}]$, and exports the

training snapshot dataset together with a repository feature cache for runtime full-history scoring. The individual steps and the intermediate files written at each stage — which make the construction process auditable and allow the dataset to be inspected before training — are listed in Appendix A.4. The final export follows the existing training snapshot contract used by OSS Risk Radar, so the historical dataset builder can feed the notebook-based training pipeline without requiring a separate model-training interface.

4.6 Observation Window and Historical Coverage

The full-history training run used in the reported evaluation contains 50,000 total rows. Of these, 41,832 rows are labeled and 8,168 rows remain unlabeled. The observation dates range from `2023-01-01` to `2025-04-01`. The labeled rows are split into 31,374 training rows, 6,275 validation rows, and 4,183 held-out test rows.

Because the label horizon is 12 months, a snapshot can serve as a labeled example only if the historical data covers its full future window; otherwise the row stays unlabeled, since the absence of future activity is meaningful only when that period is actually observed. For the thesis evaluation, event coverage must therefore be complete for the selected observation and label windows: sparse or partial GH Archive hours would make inactivity labels unreliable, because missing events could reflect incomplete coverage rather than actual inactivity. (A smaller subset is used only for local pipeline smoke testing, not for reported results.)

4.7 Feature Engineering

The feature engineering step converts repository and package history before t into numeric variables. The reported full-history model uses feature set `feature-set-v3-full-history`, which comprises 43 features spanning dependency context, commit and contributor activity, issue and pull-request flow, release cadence, age and popularity proxies, derived risk proxies, and responsiveness and backlog pressure. The feature groups, and the exact definition and rationale for every individual feature, are documented in the full feature reference in Appendix A.2 (Tables A.2 and A.3).

The features are derived from a normalized event history rather than any single source field: GH Archive supplies typed events that the builder parses into a per-repository chronological history of commits, issues, pull requests, releases, stars, and forks [28].

Each observation-time feature is a trailing-window aggregate anchored at t — counts over the 30, 90, and 365 days before t, state values at t, and derived ratios, latencies, and drop-off measures. Trailing windows are preferred to cumulative totals because the target is trajectory rather than lifetime volume, and two conventions keep variables comparable across heterogeneous projects: commit- and contributor-based counts apply a human-actor filter so that automated activity does not inflate the maintenance evidence, and absolute counts that would not compare across project sizes are also provided in a relative or trend-based form.

The selection of these signals is grounded in prior work on open-source health and survival. Revision activity has been shown to be an observable predictor of project survival [14], which motivates the commit, contributor, and release windows. The issue, pull-request, and responsiveness signals follow the observable-metric vocabulary of the CHAOSS project health model [18] and the argument that project health must be captured through structured, observable indicators rather than isolated impressions [20]. At the same time, Yehudi et al. show that individual context-free indicators are insufficient to judge sustainability across heterogeneous projects [16]; the present design responds to this critique by learning a calibrated combination of many observation-time signals rather than relying on any single metric. Responsiveness features such as issue first-response time and pull-request latency are therefore treated as reconstructed proxies rather than canonical CHAOSS metrics, in line with the finding that bot-generated responses can distort time-to-first-response measurements [15].

Some variables are lower-bound proxies rather than complete measurements: stars and forks reconstructed from GH Archive undercount history if the archive window starts after the repository was created, and reconstructed issue-backlog and pull-request response features should not be read as canonical CHAOSS metrics.

The feature set was revised after the first manual review. The revised v3 feature set makes responsiveness, stale backlog pressure, release gaps, activity drops, and contributor concentration more explicit. At the same time, features that would not be reliable point-in-time predictors are excluded from the predictive vector. For example, `repo_archived_at_obs` is retained as diagnostic metadata, while already archived observation rows are filtered out during model fitting so that archival does not become a shortcut label.

4.8 Full-History and Cold-Start Prediction Regimes

OSS Risk Radar distinguishes two prediction regimes. In the *full-history* regime, reconstructed historical activity is available from GH Archive and the repository feature cache, so the model can use time-windowed signals such as commits, releases, contributor dynamics, issue and pull-request behavior, and activity drops before the observation date. This regime is closest to the main research question — it evaluates whether historical repository signals can predict 12-month inactivity — and is the primary evaluated regime. In the *cold-start* regime, those reconstructed features are unavailable at scoring time (for example, a submitted repository not yet present in the local feature cache), so the model must rely on the public metadata obtainable through the runtime enrichment workflow.

The two regimes are not interchangeable and are not weighted equally. The full-history model is expected to perform better because it has richer temporal information, and it carries the main evaluation of the research question; the cold-start model is trained and reported as a secondary, more broadly applicable fallback, expected to be less precise and interpreted accordingly rather than as a competing result.

4.9 Leakage Prevention

Temporal leakage is a central threat in historical prediction tasks. The dataset builder applies the following rules to reduce leakage:

1. Observation-time features are derived only from timestamps $\leq t$.
2. Labels are derived only from timestamps in $(t, t + 12 \text{ months}]$.
3. Rows with incomplete future-window coverage remain unlabeled.
4. Live GitHub enrichment is limited to stable repository metadata such as creation time, default branch, and fork state.
5. Historical stars, forks, issues, pull requests, and releases are not taken from current GitHub API totals.
6. Seed strata such as active, dormant, and archived are used only for sampling provenance and not as labels.

These rules simulate a real prediction setting: at observation time t future activity is unknown, so information from after t is excluded from feature construction and used

only for label derivation.

It is important to distinguish temporal leakage from a related but legitimate effect. Several observation-time features, such as days since the last commit, the activity-drop ratio, or days since the last release, are strongly correlated with future inactivity by construction: a repository that is already quiet at t is statistically likely to remain quiet. This is not leakage, because these features use only information available at or before t. It does mean, however, that part of the predictive signal reflects activity persistence rather than the early detection of decline. This effect motivates both the trivial recency baseline and the active-at-t subset evaluation described in Section 4.11.

4.10 Model Training

The training workflow is notebook-primary. The notebook `notebooks/oss-maintenance-training.i` is used as the visible thesis workflow and as the artifact export boundary for the headless training command `npm run ml:train`. Runtime scoring is artifact-based: the scoring service expects staged model artifacts instead of retraining models during normal application use [26].

The reported learned model families are Logistic Regression, XGBoost, and a feedforward neural network, each trained for the full-history and cold-start regimes. Logistic Regression is used as an interpretable linear model, XGBoost as a strong non-linear gradient-boosted tree model, and the neural network as a non-linear model that learns feature interactions through hidden layers [21]. In addition, a trivial recency baseline is included as a non-learned reference. This baseline predicts inactivity directly from a single recency signal, such as days since the last commit or days since the last release, using a threshold selected on the validation slice. The recency baseline is necessary because the label is itself activity-related (Section 4.9); the learned models are only worth reporting if they outperform this baseline, especially on repositories that are still active at t.

The exact model configurations and hyperparameters are documented in Appendix A.3; all learned models are calibrated on the validation slice before evaluation.

Training uses a time-aware split with the default ratio:

$$75\% \text{ training}, \quad 15\% \text{ validation}, \quad 10\% \text{ test}$$

Rows are ordered by observation date before splitting. The model is fitted on the earliest

training slice. The validation slice is used for probability calibration and threshold selection. A histogram-based (binned) calibrator is fitted on the validation predictions, and a decision threshold is selected from the validation data. The final test slice is held out for reporting evaluation results. This design avoids a random split that would mix earlier and later observation periods and would not match the intended forecasting setting.

4.11 Evaluation Strategy

The primary evaluation is quantitative, on the time-aware held-out test slice, which is used neither for fitting nor for calibration. The reported metrics — accuracy, precision, recall, F1 score, Brier score, log loss, ROC-AUC, expected calibration error, and a combined quality score — are defined formally in Section A.5. Because the inactive base rate in the slice is close to balanced (approximately 0.56), ROC-AUC is a reasonable primary ranking metric [25]; precision and recall are reported at the selected operating point, but because precision depends on the positive-class prevalence and the sampled prevalence is not the expected deployment prevalence, precision and calibration are interpreted as conditional on the sampled prevalence rather than as deployment guarantees, which is why precision is not treated as a headline comparison metric [24].

Two comparisons guard against inflated headline numbers. The learned models are compared against the trivial recency baseline, and they are additionally evaluated on the subset of repositories still active at t — the harder and more decision-relevant case, because predicting that an already dormant repository stays inactive is close to trivial. Reporting this subset separately prevents activity persistence (Section 4.9) from dominating the overall metrics.

This quantitative evaluation is complemented by manual case review, because inactivity risk is an operational proxy rather than a directly observable truth; such review identifies cases where the model is right for the wrong reason or where context demands caution, and it drove the feature-set revision in the first iteration. The held-out split is a valid *internal* evaluation of temporal generalization within the constructed dataset, but not of generalization to a freshly sampled repository set, which is left to future work and discussed as a threat to external validity in Chapter 8.

The primary deployed full-history artifact is `xgboost-full-history`, version `0.2.0`, trained with `feature-set-v3-full-history`; it is the model staged for runtime scoring. Its held-out test metrics, together with those of the Logistic Regression model, the neural

network (`neural-net-full-history`), and the trivial recency baseline, are reported and interpreted in Chapter 6.

4.12 Methodological Limitations

This section records limitations that are intrinsic to the method and its data. Broader validity threats and their mitigation are consolidated in Chapter 8; this section is intentionally restricted to data and design limitations.

First, the method depends on the completeness of the provided GH Archive files. If relevant event hours are missing, repository histories may be incomplete. Second, public GitHub activity does not capture all possible maintenance activity. Some projects may move development to other platforms, publish releases outside GitHub, or maintain stable software with little visible activity. Third, package-to-repository mappings can be ambiguous, especially when packages move repositories or when metadata is incomplete. Fourth, the inactivity label is a proxy for elevated maintenance risk and not a definitive statement that a project is abandoned or insecure, and the label thresholds described in Section 4.3 are heuristic design choices whose effect on prevalence and results is a known robustness concern.

The foundation seed also bounds the scope and prevalence of the results. Because the dataset targets repositories with a license object and a minimum star threshold, the evaluation covers visible open-source repositories rather than all public GitHub repositories; and, as explained in Section 4.4, the deliberate oversampling of dormant and archived repositories means predicted probabilities and precision are conditional on the sampled prevalence and should be recalibrated before being read as deployment-time risk estimates.

Finally, the current internal test split evaluates temporal generalization within the constructed dataset. This is useful and valid for model development, but it does not replace external validation on a newly sampled repository set. For this reason, external validation on repositories outside the foundation dataset is identified as future work in Chapter 9 rather than claimed as a completed result.

The next chapter describes the implementation of OSS Risk Radar and explains how the dataset builder, model artifacts, scoring service, backend, and frontend are integrated into the overall artifact.

5 Implementation

This chapter describes how the methodological design of OSS Risk Radar was realized in software. It concentrates on the parts that bear on the research question: the overall system architecture, the leakage-controlled historical dataset builder, the two dataset-construction corrections that emerged from the design cycle, and the instrumentation that supports the reported evaluation. Engineering detail that is needed only to reproduce or operate the artifact — the language and technology choices, the dataset-builder module layout, the model configurations, the artifact format, and the scoring service, backend, frontend, and deployment — is collected in Appendix A.3 so that it does not crowd out the scientifically relevant decisions. The artifact remains reproducible and auditable, consistent with the Design Science Research framing of the thesis [11], [12].

5.1 System Architecture Overview

OSS Risk Radar is organized as a single repository (a monorepo) that contains several cooperating components (a Python scoring-and-training workspace, a Go backend API, a TypeScript/Next.js analyst frontend, shared schemas, Node.js orchestration scripts, and Docker/Kubernetes deployment assets; their responsibilities are listed in Appendix A.3, Table A.4). This structure keeps the data contracts, the training code, and the runtime services in one versioned location, which is important because the same feature definitions must be applied both during offline training and during online scoring.

The runtime data flow follows the methodology. An analyst submits one or more repositories through the frontend; the Go backend registers an analysis and enqueues a durable background job in PostgreSQL; a worker enriches each repository with public signals from GitHub and OpenSSF Scorecard and calls the Python scoring service with staged model artifacts, which returns an inactivity-risk profile that the frontend renders [26]. The unit of analysis is a single repository; the dependency inventory to grade is supplied by an external software-composition-analysis tool such as the OSS Review

Toolkit [17], so the artifact itself does not resolve dependency graphs.

A central design principle is the strict separation between offline training and online scoring. Models are never trained during normal application use. Instead, the training pipeline exports versioned model artifacts that are promoted into a staged bundle, and the scoring service loads only these artifacts at runtime. Missing artifacts are treated as a deployment configuration error rather than as a reason to fall back to substitute scoring behavior. This separation guarantees that the deployed model is exactly the model that was evaluated for the thesis.

5.2 Historical Dataset Builder

The historical dataset builder is implemented as a Python package under `app/training/maintenance_`
It transforms a repository seed and raw GH Archive event files into an inspectable, leakage-controlled training dataset, and is divided into focused modules for ingestion, event parsing, feature computation, and label computation so that each stage can be tested independently; the module layout is documented in Appendix A.3.

The separation between observation-time and future-window computation is enforced in code. Each snapshot fixes its feature window to the year before t, a previous window to the year before that, and a label window of twelve months after t. Feature computation only consults events at or before the observation date, and label computation only consults events strictly after it. The label implementation reflects the two-of-four maintenance rule directly: it counts the number of distinct months with human commits, the number of distinct future contributors, the number of future releases or published package versions, and the number of merged pull requests, and marks a repository as maintained only if it is not archived or deleted within the horizon and at least two of these conditions are met. When the dataset-wide archive coverage horizon does not span the full label window, the row is marked with a `future_window_incomplete` signal and left unlabeled, so that missing data is never silently interpreted as inactivity. The granularity at which this coverage horizon is measured was itself the subject of a correction, discussed in Section 5.2.1.

Human-versus-bot attribution is applied during both feature and label computation. Commit and contributor counts use an actor filter that excludes automated accounts, so that bot-driven activity does not inflate maintenance evidence. This is consistent with the cautious treatment of responsiveness signals motivated in the methodology, where automated responses are known to distort interaction metrics [15].

The export step produces two outputs with different purposes. The training snapshot file follows the existing training contract and feeds the model-training pipeline. The repository feature cache stores, for each repository, the most recent observation-time feature row, with diagnostic-only fields such as `repo_archived_at_obs` removed. This cache is what later allows the runtime backend to score a previously seen repository in the richer full-history regime rather than the cold-start regime.

Two design-cycle refinements of the builder are worth recording. First, the feature set was revised after the formative review to replace raw counts that were poor point-in-time predictors (such as an absolute open-issue count, which large healthy projects also carry) with relational and trend-based variables, and to resolve a single primary package-to-repository link per repository so that mirror repositories do not make an actively developed project look inactive; the details are given in Appendix A.3.12. Second, and more consequential for the labels, is the completeness correction described next.

5.2.1 Label-Completeness Correction

A further design-cycle iteration corrected how label completeness is determined. The prediction objective, the label window $(t, t{+}12$ months], and the two-of-four maintenance rule are unchanged; the iteration concerns only which observation rows are admitted as labeled examples, and is therefore a dataset-construction correction rather than a change of the model or its target.

The methodology requires that a row be labeled only when the historical data actually covers its full future window, precisely because the absence of future activity is meaningful only when the future period is observed (Section 4.9). The initial implementation enforced this using each repository's own last observed event as its coverage end. This proxy is biased in a way that is correlated with the outcome: a repository that simply falls quiet has, by definition, no late events, so its coverage end is early and its later observation rows were discarded as "incomplete", even when the archive fully covered the corresponding window. The very repositories that exhibit the inactivity the model is meant to detect were thus systematically under-represented among the labeled positives. The symptom was visible in the smoke fixtures, where a synthetic late "coverage marker" event had to be injected into an otherwise inactive repository purely to keep its rows labelable.

The corrected implementation gates completeness on a dataset-wide archive coverage horizon rather than on any single repository's last event. This horizon is the latest

event observed across all repositories in the run, or an explicit cutoff supplied through the new `--coverage-end` option, which the orchestration script defaults to the latest day of complete hourly GH Archive coverage. A repository that is silent but still inside this horizon is now labeled inactive, as intended, while rows whose window genuinely extends past the available data remain unlabeled. For transparency, a row whose own last event precedes the horizon but which remains labelable is annotated with a `repo_silent_before_horizon` diagnostic signal, so that the share of rescued silent repositories can be reported as part of the dataset-quality summary. This change aligns the implementation with the leakage-prevention rule already stated in the methodology and removes a selection effect that previously depressed the labeled inactive population.

5.3 Model Training and Evaluation Instrumentation

OSS Risk Radar implements three learned model families for both the full-history and the cold-start regimes: Logistic Regression and a feedforward neural network, both implemented from first principles so that the artifact stays dependency-light and fully inspectable, and XGBoost through its established library. All three share the same dataset, the same time-aware split, and the same calibration and evaluation code; the exact model configurations and the artifact format are documented in Appendix A.3.

All three families produce raw probabilities that are post-processed identically. A histogram (binned) calibrator is fitted on the validation predictions — grouping them into probability bins, taking each bin's smoothed empirical inactivity rate, and enforcing a monotonic mapping — and is stored with the artifact and re-applied at inference, so the runtime service reproduces exactly the calibration that was evaluated. A decision threshold is then selected on the calibrated validation predictions by maximizing the F1 score. The evaluation module computes the reported metrics (Section 4.11) on the held-out test slice. Two evaluation instruments underpin the reported figures. A *slice evaluation* re-scores the same held-out predictions within meaningful subgroups (popularity tier, repository-age band, cold-start-like versus has-history regime, and the active-at-t subset), so that a strong majority subgroup cannot mask a weak minority one; it reuses the same metric code and a consistency check ties the per-slice figures to the reported headline ROC-AUC. A *repository-disjoint split* additionally assigns every snapshot of a repository to a single partition, so the held-out set contains only repositories absent from training; comparing it against the default chronological split bounds the optimism introduced by repository recurrence. In practice the held-out ROC-AUC decreases only marginally under the repository-disjoint split (Section A.3.14), which indicates that the model's discrimination is not an artifact of memorizing individual

repositories. Both instruments leave the serialized artifacts and headline metrics unchanged; their mechanics are documented in Appendix A.3.13 and Appendix A.3.14, and the per-split and per-slice figures are reported in Chapter 6.

5.4 From Training to Deployment

The training workflow is notebook-primary: the notebook `notebooks/oss-maintenance-training.ip` is the visible thesis workflow and the artifact export boundary, and the headless command `npm run ml:train` executes it non-interactively to produce the run artifacts. Trained artifacts are not deployed automatically. A staging command validates a candidate bundle and promotes it into the deployed bundle under `deployment/training` only if it passes a quality gate whose most important condition is a regression comparison against the currently deployed baseline: for each required model, ROC-AUC must not drop and the Brier score must not increase by more than a configured tolerance. As a result, the deployed model can only ever improve or stay within tolerance relative to the previously evaluated baseline. The artifact is containerized and, for production-style operation, deployed to a k3s cluster managed by Argo CD; because the deployed model bundle is a versioned set of files, deployment and model promotion remain auditable, and the running service uses exactly the artifacts that passed the staging gate. The staging gate, the scoring service, the backend orchestration, the frontend, and the deployment setup are described in full in Appendix A.3.

The next chapter reports and discusses the empirical results obtained with this implementation.

6 Results and Discussion

This chapter reports and interprets the empirical results of the trained artifacts: predictive performance against baselines, calibration behaviour, feature attribution and error analysis, and an ablation of the human-actor filter.

6.1 Predictive Performance

Table 6.1 reports held-out test metrics for the three learned model families in both feature regimes, together with the trivial recency baseline. All learned models were trained on the same time-aware split (75/15/10; $n_{\text{test}} = 4{,}183$ labelled observations) and calibrated on the validation slice with the same histogram calibrator, so the comparison isolates the effect of model family rather than data or calibration differences. The combined quality score summarizes ranking skill (ROC-AUC) and calibration (Brier) into a single comparison figure and is not intended as a standalone proof. All reported metrics are defined formally in Section A.5.

Table 6.1: Held-out test metrics by model family and feature regime. Higher is better for ROC-AUC, F1, and quality; lower is better for Brier and ECE. The best value within each regime is shown in bold.

Model	ROC-AUC	Brier	ECE	F1	Quality
`logistic-regression-full-history`	0.930	0.097	**0.012**	0.883	0.759
`xgboost-full-history`	**0.945**	**0.089**	0.016	**0.891**	**0.790**
`neural-net-full-history`	0.939	0.092	0.013	0.888	0.778
`logistic-regression-cold-start`	0.885	0.127	0.049	0.857	0.656
`xgboost-cold-start`	**0.940**	**0.093**	**0.011**	**0.887**	**0.777**
`neural-net-cold-start`	0.937	0.095	**0.011**	0.884	0.771
`recency-baseline` (days since commit)	0.655	—	—	0.718	—

All three families rank inactivity well above chance in both regimes. In the full-history regime the gradient-boosted trees (`xgboost-full-history`) achieve the strongest ranking and calibration (ROC-AUC 0.945, Brier 0.089), ahead of the multilayer perceptron

(0.939 / 0.092) and logistic regression (0.930 / 0.097); logistic regression attains the lowest expected calibration error but at a clear cost in ranking skill. The cold-start regime preserves the same ordering, with the gap between the linear model and the two non-linear models widening (logistic regression falls to ROC-AUC 0.885 and Brier 0.127), which is consistent with non-linear models exploiting interactions among the reduced cold-start feature set.

The neural network was included as a deliberate baseline to test whether a higher-capacity non-linear model is warranted for this tabular feature space. It does not improve on the gradient-boosted trees on any reported metric in either regime, while adding training cost, weaker interpretability, and additional calibration sensitivity. This is consistent with the broader finding that tree-based models remain a strong default on moderate-sized tabular problems [30]. Accordingly, logistic regression is retained for its transparent coefficient attribution and XGBoost as the deployed runtime scorer, while the neural network is reported as a tested-and-rejected alternative rather than promoted into the scoring ensemble.

Two runtime compositions of these evaluated models are reported as engineering mechanisms rather than as separately benchmarked results. At scoring time the backend averages the staged models of a regime into an ensemble profile and attaches a model-agreement factor, and it derives a per-prediction confidence from signal completeness and prediction decisiveness. Both are exercised in the demonstrator (Chapter 7); the headline metrics in this chapter are, however, always reported for the individual artifacts, so no claim is made that the runtime ensemble outperforms its strongest member. Quantifying a possible ensemble gain is left to future work (Chapter 9).

The learned models are worth reporting only if they improve on the trivial recency signal that the label is partly built from (Section 4.10). The last row of Table 6.1 reports this baseline: ranking the held-out test slice purely by days since the last commit attains a ROC-AUC of only 0.655, and selecting an F1-optimal single threshold on the validation slice degenerates to predicting inactivity for essentially every repository (threshold at zero days, recall 1.0, F1 0.718 at the balanced base rate). Every learned model clears this bar by a wide margin — the deployed `xgboost-full-history` model raises ROC-AUC from 0.655 to 0.945 — so the models add substantial ranking skill over recency alone rather than merely restating it. To further separate genuine predictive skill from activity persistence, the deployed full-history model is additionally re-scored on the active-at-t subset defined in Section 4.11; this slice analysis is reported in Section 6.3, where the same recency baseline is also revisited.

6.2 Calibration and Threshold Selection

Because the score is intended to be read as a probability rather than only as a ranking, calibration is evaluated alongside discrimination. Every model's raw outputs are mapped through a histogram calibrator fitted on the validation slice and never on the test slice, so the reported calibration reflects held-out behaviour. For the deployed `xgboost-full-history` model this yields a Brier score of 0.089, a log loss of 0.284, and an expected calibration error of 0.016 at the default decision threshold of 0.5, where the operating point attains precision 0.87 and recall 0.92. The linear `logistic-regression-full-history` model reaches the lowest expected calibration error (0.012) but a weaker Brier score (0.097), consistent with its lower ranking skill in Table 6.1.

The reliability of the calibrated probabilities is well behaved: the empirical inactivity rate rises monotonically across the calibration bins, from approximately 0.02 in the lowest predicted-probability band to 0.98 in the highest, so higher scores do correspond to higher observed inactivity frequencies rather than only to a higher rank. Two caveats apply. First, the two extreme bands hold most of the held-out mass, so mid-range probabilities are supported by fewer samples and are correspondingly less certain; the per-prediction confidence surfaced by the demonstrator (Chapter 7) reflects exactly this bin-level support. Second, calibration is conditional on the sampled prevalence rather than a natural deployment prevalence (Section 4.12), so the absolute probabilities would require recalibration before being read as live population probabilities.

6.3 Feature Importance and Error Analysis

6.3.1 Feature Attribution

Feature attribution is read from the logistic-regression model, which is retained precisely for its transparent coefficients (Section 6.1). Because the inputs are standardized, the coefficient magnitudes are directly comparable, and their signs indicate the direction of the association with inactivity. The strongest inactivity-*lowering* signals are the breadth of sustained development (`active_commit_months_365d`, standardized coefficient -0.94) and recent maintainer integration (`has_recent_pr_merge_flag`, -0.47); the strongest inactivity-*raising* signals are staleness and slow responsiveness, including days since the last commit and last push (each $\approx +0.19$), a slower median issue first response ($+0.16$), and a larger stale-issue share ($+0.15$). These directions match the intended semantics of

the features (Appendix A.2). A few coefficients are less intuitive (for example a positive association for the log-scaled star count), which reflects partial associations under the sampled prevalence and correlations among features rather than a causal effect; the coefficients are therefore used to explain *direction* for a given repository, while ranking performance is carried by the gradient-boosted model.

The gradient-boosted model exposes a complementary, non-linear view through its gain-based feature importances (normalized to sum to one over the 39 features that receive at least one split). Its ranking is dominated by recent commit volume: `commits_365d` (0.29) and `commits_90d` (0.26) together account for over half of the total gain, followed by `recent_contributors_90d` (0.12) and `active_commit_months_365d` (0.05); pull-request and issue-flow features (`opened_prs_90d`, `open_issue_growth_90d`, `merged_prs_90d`), popularity (`stars_log1p`), and release activity (`releases_365d`) contribute the next tier. The two views agree on the substantive story — sustained recent development activity and contributor breadth are the strongest signals — while differing in emphasis: the linear model leans on the breadth of active commit months, whereas the gradient-boosted model concentrates gain on the raw recent-commit-volume windows.

6.3.2 Robustness to Repository Recurrence

Under the repository-disjoint split (Section A.3.14), where every snapshot of a repository is confined to one partition, held-out ROC-AUC decreases only from 0.945 to 0.940 and the Brier score rises only from 0.089 to 0.090. The small gap bounds the optimism attributable to a repository appearing in both training and test slices and indicates that the discrimination generalizes to unseen repositories rather than resting on repository memorization.

6.3.3 Slice Evaluation

A single aggregate metric can average a strong subgroup with a weak one. The held-out predictions of the deployed `xgboost-full-history` model are therefore re-scored within subgroups that vary in ways relevant to deployment; the full per-subgroup table is given in Appendix A.8 (Table A.8). Three findings follow. First, the aggregate ROC-AUC masks a much weaker obscure-repository subgroup: low-popularity repositories are ranked at only 0.689, against 0.968 for high-popularity ones, so scores for low-visibility projects deserve more caution and are exactly where the per-prediction confidence should be low. Second, and most importantly for the activity-persistence threat (Section 8.2), the model still separates outcomes well *among repositories that are*

active at the observation time (ROC-AUC 0.883 on the active-at-t subset, where the inactive rate is only 0.17); the headline metric is therefore not merely a restatement of "already-quiet repositories stay quiet". On this same active-at-t subset the trivial recency baseline reaches only ROC-AUC 0.777, so the learned model retains a clear margin over recency exactly where early-warning skill matters most. Third, full history helps: repositories with reconstructable history are ranked at 0.917 versus 0.872 for cold-start-like repositories, consistent with the two-regime design (Section 4.8).

6.3.4 Error Analysis

The comparison against an explicit trivial recency baseline is now part of the reported results (Section 6.1 and the slice analysis above); the systematic formative case review that motivated the feature-set revision (Section 4.2) remains a complementary qualitative analysis and is noted as a remaining item in Chapter 9. The slice results above already give the main error-analysis conclusion: the dominant residual weakness is low-visibility repositories with sparse public signals, which the confidence layer is designed to flag rather than hide.

6.4 Ablation: The Human-Actor Filter

Commit- and contributor-based features and the maintenance label are computed from human actors only; automated (bot) accounts are excluded (Section 4.7). Because this is a design choice that could plausibly be omitted, it is tested directly by an ablation: the full 5,000-repository dataset was rebuilt with the human-actor filter disabled through a single build flag, with every other setting identical, yielding 50,000 snapshots that align one-to-one with the filtered build. Three effects are examined.

Label corruption. Disabling the filter flips 357 of 50,000 labelled snapshots (0.71%) from inactive to active, and *none* in the opposite direction. The effect is strictly one-directional: a bot commit or merge inside the future window can satisfy the two-of-four maintenance rule, so an otherwise dormant repository is relabelled as maintained, whereas bots never make an active repository look inactive. Omitting the filter would therefore bias the target by systematically *masking* inactivity, which is the error direction that matters most for a risk signal.

Feature inflation. On the observation-time side, bots inflate the activity features substantially: with the filter disabled, `commits_365d` changes on 7,858 of 50,000 snapshots (15.7%) by a mean of +938 commits, `commits_90d` by +388, and `contributors_365d`

by +1.4 distinct contributors, while pull-request *count* features are unaffected because they are not actor-filtered. A small number of bot-driven repositories would thus dominate the activity signal.

Predictive effect. Retraining all four model families on the unfiltered dataset changes the held-out metrics only at the noise level ($|\Delta\text{ROC-AUC}| \leq 0.003$, $|\Delta\text{Brier}| \leq 0.002$); measured against each dataset's own labels, bots appear harmless. This is expected but misleading, because the unfiltered model is then scored against a partially corrupted target. A cleaner test trains on the bot-contaminated *features* while evaluating against the clean, human-filtered *labels* (Appendix A.8, Table A.9): the contaminated features are consistently worse across all four families, though by a small margin (ROC-AUC falls by 0.0007–0.0018; Brier rises by up to 0.0018).

Interpretation. The filter's justification is construct validity rather than discrimination. Its predictive cost when omitted is small, since the gradient-boosted models are largely robust to the minority of bot-inflated repositories, but it is consistently in the expected direction, and, more importantly, the filter removes a one-directional label bias that a headline metric computed on the corrupted data does not reveal. The human-actor filter is therefore retained. Absolute metrics in this ablation differ slightly from Table 6.1 because the ablation uses a freshly rebuilt, fully labelled dataset; the comparison is internally controlled and varies only the filter.

7 Practical Demonstrator

This chapter demonstrates how the trained inactivity-risk model is applied end to end, from a set of dependency repositories to a ranked risk view that supports triage. While Chapter 6 evaluates the model on the held-out test set, this chapter shows the artifact operating on a realistic input, as required by the demonstration phase of the Design Science Research process [11], [12]. The demonstrator is run against the deployed OSS Risk Radar instance using the staged model bundle (primary full-history artifact `xgboost-full-history`, version `0.2.0`); all figures reported here are real outputs of that deployed artifact and reflect the public repository state at the time of analysis.

7.1 Example Repository Set

The unit of analysis is a single repository, and each repository is scored independently. Enumerating which repositories a project depends on is the task of established software-composition-analysis tooling, such as the OSS Review Toolkit [17]; the artifact treats this dependency inventory as an external input rather than resolving dependency graphs itself.

For this demonstration the input is the set of dependency repositories of `ArchipelagoMW/Archipelago`, a large and actively developed open-source multi-world game randomizer. Its declared Python dependencies correspond to 30 source repositories, ranging from widely used infrastructure libraries (for example `flask`, `jinja2`, `werkzeug`) to smaller, more specialized packages. This project was chosen because that mixture is expected to produce a non-trivial triage view rather than a uniformly low-risk one.

Each repository is scored through the deployed analysis workflow, which enriches live repository metadata and applies the staged model ensemble. The two prediction regimes of Section 4.8 appear directly in this run: one repository (`flask`) is present in the staged repository feature cache and is therefore scored in the richer full-history regime, while the remaining 29 are absent from the foundation cache and fall back to the cold-start regime. This distribution is itself an honest illustration of the cold-start coverage

limitation highlighted in Chapter 9: an arbitrary project's dependency repositories are mostly outside the precomputed foundation seed and are consequently scored by the weaker cold-start model, a fact the confidence layer is designed to surface rather than hide.

7.2 Risk Ranking Output and Interpretation Guidance

The scoring service produces, for each repository, the inactivity-risk score (0–100), the discrete action level (`monitor`, `review`, or `replace_candidate`), a confidence value derived from signal completeness and prediction decisiveness, and the prediction regime. Of the 30 scored repositories, the distribution is one high-risk item (action `review`), five medium-risk items, and 24 low-risk items; all but one are assigned the `monitor` action, so the ranking concentrates analyst attention on a single repository rather than diluting it across the whole set. Table 7.1 shows the flagged and medium-risk repositories together with the lowest-risk full-history item for contrast; the complete 30-repository ranking is given in Appendix A.7.

Table 7.1: Highest-risk dependency repositories of `ArchipelagoMW/Archipelago` (the `review` item and the five medium-risk items), with the lowest-risk full-history repository shown for contrast. Risk is the 0–100 inactivity-risk score; "Conf." is the per-prediction confidence; "Regime" is full-history (FH) or cold-start (CS). The complete 30-repository ranking is in Appendix A.7.

Package	Source repository	Risk	Action	Conf.	Regime
bsdiff4	`ilanschnell/bsdiff4`	77.9	review	0.69	CS
cymem	`explosion/cymem`	46.6	monitor	0.81	CS
kivymd	`kivymd/KivyMD`	46.6	monitor	0.71	CS
pymem	`srounet/Pymem`	46.6	monitor	0.71	CS
markupsafe	`pallets/markupsafe`	46.6	monitor	0.81	CS
setproctitle	`dvarrazzo/py-setproctitle`	46.6	monitor	0.67	CS
flask	`pallets/flask`	1.7	monitor	0.97	FH

The flagged repository. The single item that the ranking elevates for review is `bsdiff4` (`ilanschnell/bsdiff4`), a small binary-diff library with a risk score of 77.9. Its explanation factors, taken directly from the scored output, are consistent with the intended semantics of the model: the two cold-start ensemble members predict a 78.7% and 77.1% inactivity risk respectively, the model-agreement factor reports a narrow spread between them, and the input-completeness factor records that 88% of expected signals were available before imputation. The score reflects a genuinely quiet

maintenance profile (a low-popularity repository with no contributor activity in the recent window and no detected release cadence), which is exactly the kind of small, single-maintainer dependency that a supply-chain reviewer would want surfaced. The confidence of 0.69 is moderate rather than high, and a caveat records that the score was produced by the cold-start model because no full-history cache row was available, so the flag is presented as a prompt for manual review rather than as a definitive judgement.

A low-risk repository for contrast. At the opposite end, `flask` (`pallets/flask`) receives a risk score of 1.7 with a confidence of 0.97. It is the one repository scored in the full-history regime, and both full-history ensemble members agree on a 1.7% inactivity risk with 98% signal completeness. This contrast between a moderate-confidence cold-start flag and a high-confidence full-history clearance illustrates why the confidence value, not only the risk score, is part of the triage output.

Interpretation guidance follows the rest of the thesis. The score is a prioritization signal, not a trust or security verdict: a low score does not certify a repository as secure, and the elevated score for `bsdiff4` indicates thin public maintenance signals rather than a known defect. Low confidence or a large number of imputed signals should reduce the decisiveness of any action, and cold-start scores in particular are conditional on the reduced feature set and on the sampled prevalence to which the model is calibrated (Section 4.12), so they warrant recalibration before being read as absolute deployment probabilities. Used this way, the ranked view converts a flat list of 30 repositories into a single reviewable candidate plus a small medium-risk cluster, which is the intended practical contribution of the artifact.

8 Threats to Validity

This chapter consolidates the threats to validity for the thesis and the mitigations applied. It is the single home for validity discussion; the methodology chapter (Section 4.12) is restricted to intrinsic data and design limitations, which are referenced here rather than repeated. Threats are organized along the standard categories of construct, internal, external, and conclusion validity.

8.1 Construct Validity

Construct validity concerns whether the measured quantities represent the intended concepts. The central construct is repository inactivity, which is operationalized as the inverse of a multi-signal maintenance rule over the future window $(t, t + 12 \text{ months}]$ (Section 4.3). This is a proxy: a repository can be stable and low-activity without being abandoned, and visible activity does not capture development that moves to other platforms or release channels. The two-of-four maintenance thresholds are heuristic design choices, and the merged-pull-request clause is a proxy for broader maintainer interaction. These choices are mitigated by requiring sustained rather than incidental activity and by interpreting the score as a triage signal rather than a verdict, but they remain a construct-validity threat that a label sensitivity analysis (Chapter 9) would further address. A related construct decision — excluding automated (bot) actors from the activity features and the label — is validated directly by the ablation in Section 6.4: disabling the filter flips 0.71% of labels from inactive to active and never the reverse, confirming that the filter removes a one-directional bias that masks inactivity rather than merely improving model fit.

8.2 Internal Validity

Internal validity concerns whether the reported predictive relationship is genuine rather than an artifact of the experimental setup. The main threat is temporal leakage,

mitigated by the leakage-prevention rules in Section 4.9: observation-time features use only timestamps $\leq t$, labels use only the future window, rows with incomplete future coverage remain unlabeled, and live enrichment is restricted to stable repository metadata. A distinct, non-leakage threat is activity persistence: features such as days since last commit are predictive of near-term inactivity by construction, so part of the discrimination reflects already-quiet repositories staying quiet. This is mitigated by the active-at-t subset evaluation (Section 6.3.3): on repositories that are still active at the observation time, where the inactive rate is only 0.17, the model still attains ROC-AUC 0.883, so the headline metrics are not credited to persistence alone. A comparison against an explicit trivial recency baseline is a complementary check that is identified as a remaining item (Chapter 9).

A second internal-validity concern is repository recurrence across the chronological split. Because observations are quarterly, a repository contributes several snapshots with overlapping label windows, and the chronological split can place earlier snapshots of a repository in training and later ones in the held-out set; given the autocorrelation between a repository's snapshots, this makes the reported metric optimistic relative to performance on previously unseen repositories. This threat is addressed by additionally evaluating the model under a repository-disjoint split, in which every snapshot of a repository is confined to a single partition (Section A.3.14). The held-out ROC-AUC decreases only from 0.945 to 0.940 under the repository-disjoint split, which bounds the optimism attributable to repository recurrence and indicates that the discrimination generalizes to unseen repositories rather than resting on repository memorization.

8.3 External Validity

External validity concerns generalization beyond the studied sample. Two threats apply. First, the foundation seed targets repositories with a license object and a minimum star threshold, so results should not be generalized to very small, private, newly created, or low-visibility repositories. Second, the seed deliberately oversamples dormant and archived repositories, so the inactive base rate in the dataset (approximately 0.56 in the test slice) does not reflect the natural OSS inactivity rate; predicted probabilities and precision are conditional on this sampled prevalence and would require recalibration for a deployment population (Section 4.12). The held-out split evaluates temporal generalization within the constructed dataset but not generalization to a freshly sampled repository set; external validation on out-of-foundation repositories is identified as future work (Chapter 9).

8.4 Conclusion Validity

Conclusion validity concerns whether the conclusions drawn from the metrics are warranted. The evaluation uses a single time-aware held-out split rather than repeated resampling, so reported metrics are point estimates without confidence intervals. Because prevalence is sampled rather than natural, precision-dependent conclusions are reported as conditional on the sampled prevalence, and ROC-AUC is used as the primary ranking metric with precision-recall analysis as a complement (Section 4.11). Qualitative case review is used as supporting, not confirmatory, evidence and is interpreted as formative.

9 Conclusion and Outlook

This thesis asked to what extent publicly observable repository health and maintenance indicators can predict 12-month OSS dependency inactivity, and how the resulting inactivity-risk score can be combined with parallel security-practice signals to support dependency triage. The empirical results support a qualified positive answer. On a time-aware held-out split, the deployed full-history gradient-boosted model ranks inactivity with a ROC-AUC of 0.945 and is well calibrated (Brier 0.089, expected calibration error 0.016). The skill largely persists under a repository-disjoint split (0.940), so it generalizes to unseen repositories rather than memorizing them, and it remains substantial among repositories that are active at the observation time (0.883), so it is not merely a restatement of activity persistence.

At the same time, the model is markedly weaker on low-visibility repositories (ROC-AUC 0.689) and in the cold-start regime, and its probabilities are calibrated to a sampled rather than a natural prevalence. The score is therefore presented as a calibrated triage aid with an explicit per-prediction confidence that flags exactly these weak cases, not as a verdict on abandonment. Consistent with the research design, security-practice signals are surfaced as complementary triage context and are deliberately excluded from the inactivity model, so the thesis does not claim that they improve inactivity prediction. Applied per repository to the dependency repositories of a project, whose inventory is supplied by an external software-composition-analysis tool such as the OSS Review Toolkit [17], these scores produce the ranked, evidence-backed triage view demonstrated in Chapter 7, which is the intended practical contribution.

9.1 Outlook and Future Work

Several extensions are deliberately scoped out of the completed method and are recorded here as future work rather than as claimed results.

- **External validation.** The reported evaluation uses a time-aware held-out split within the constructed foundation dataset (Section 4.11). A stronger robustness

check would re-sample a fresh set of repositories that were not part of the foundation seed and evaluate the trained artifacts on that independent set. This would test generalization beyond the sampled prevalence and seed strata.

- **Systematic qualitative error analysis.** The completed artifacts cover Logistic Regression, XGBoost, and a feedforward neural network; the active-at-t slice isolates predictive skill from activity persistence (Section 6.3.3), and the trivial recency baseline is now reported and clearly outperformed (Section 6.1). A systematic, larger-scale qualitative case review — beyond the formative examples that motivated the feature-set revision — remains to be added as complementary evidence.

- **Additional model families.** Beyond the covered families, further extensions such as more expressive deep-learning architectures or sequence models over the repository event history are left as future work.

- **Ensemble evaluation.** The runtime averages the staged models of a regime into an ensemble profile (Section 6.1), but the reported metrics are for the individual artifacts. Benchmarking whether the ensemble improves on its strongest member, and whether the model-agreement spread is a useful uncertainty signal, is left as future work.

- **Prevalence-aware calibration.** Because the seed oversamples dormant and archived repositories, predicted probabilities are calibrated to the sampled prevalence (Section 4.12). Recalibrating to an estimated deployment prevalence would make the scores directly usable as live triage probabilities.

- **Label sensitivity analysis.** The maintenance label uses heuristic thresholds (Section 4.3). A systematic sensitivity analysis over these thresholds would quantify how strongly the results depend on the specific label definition.

- **Cold-start coverage and on-demand history reconstruction.** Repositories outside the foundation seed are scored by the cold-start model, which uses only features derivable from a single live snapshot and therefore omits the strongest historical signals (most notably the distribution of commit activity across the year). Two observations make this a priority for future work. First, the held-out cold-start metrics are measured on seed-distribution repositories but applied to arbitrary submissions, so the deployed cold-start performance is likely optimistic; the per-prediction confidence already flags such out-of-distribution cases rather than hiding them. Second, a submitted repository's recent history can in principle be reconstructed on demand from the GitHub REST API (for example, weekly

commit activity and per-contributor statistics) to recover several full-history features. This was deliberately not implemented, because the live API cannot supply the full feature set: package-registry features (package age, dependency count, published versions, popularity tier) are not exposed by the repository API, per-issue first-response latency requires prohibitively many timeline requests, and the year-over-year change features require a second year of history. Imputing this missing tail into the full-history model would itself introduce a distribution shift. A cleaner extension is therefore a third *warm-start* model trained on exactly the subset of features that can be reconstructed reliably from the live API, validated for parity against the precomputed seed feature cache and supported by a rate-limit-aware request cache. This is left as future work because of the additional engineering and validation effort it requires.

Bibliography

[1] Open Source Initiative, *The open source definition*, [Online]. Available: https://opensource.org/osd Accessed: 2026-02.

[2] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-218, Feb. 2022. DOI: 10.6028/NIST.SP.800-218

[3] SLSA Community, *Slsa specification v1.2*, [Online]. Available: https://slsa.dev/spec/v1.2/, Nov. 2025.

[4] C. Miller, M. Jahanshahi, A. Mockus, B. Vasilescu, and C. Kästner, "Understanding the response to open-source dependency abandonment in the npm ecosystem," in *Proc. IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2355–2367. DOI: 10.1109/ICSE55347.2025.00004

[5] C. Miller, C. Kästner, and B. Vasilescu, ""we feel like we're winging it:" a study on navigating open-source dependency abandonment," in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1281–1293. DOI: 10.1145/3611643.3616293

[6] G. A. A. Prana et al., "Out of sight, out of mind? how vulnerable dependencies affect open-source projects," *Empirical Software Engineering*, vol. 26, no. 4, 2021. DOI: 10.1007/s10664-021-09959-3

[7] npm, Inc., *Kik, left-pad, and npm*, [Online]. Available: https://blog.npmjs.org/post/141577284765 left-pad-and-npm, Mar. 2016.

[8] Equifax, *Equifax releases details on cybersecurity incident*, [Online]. Available: https://investor.equifax.com/news-events/press-releases/detail/237/equifax-releases-details-on-cybersecurity-incident, Sep. 2017.

[9] Federal Trade Commission, *Equifax to pay $575 million as part of settlement with ftc, cfpb, and states related to 2017 data breach*, [Online]. Available: https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach, Jul. 2019.

[10] Open Source Security Foundation and OpenJS Foundation, *Open source security and openjs foundations issue alert for social engineering takeovers of open source projects*, [Online]. Available: https://www.linuxfoundation.org/blog/openssf-openjs-alert-for-social-engineering-takeovers-of-open-source-projects, Apr. 2024.

[11] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302

[13] GitHub, *Rest api endpoints for licenses*, [Online]. Available: https://docs.github.com/en/rest/lice
Accessed: 2026-02.

[14] D. Robinson, K. Enns, N. Koulecar, and M. Sihag, *Two approaches to survival analysis of open source python projects*, arXiv:2203.08320, 2022.

[15] K. A. Hasan, M. Macedo, Y. Tian, B. Adams, and S. H. H. Ding, "Understanding the time to first response in github pull requests," in *Proc. IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023.

[16] Y. Yehudi, C. Goble, and C. Jay, *Individual context-free online community health indicators fail to identify open source software sustainability*, arXiv:2309.12120, 2023.

[17] OSS Review Toolkit, *Analyzer*, [Online]. Available: https://oss-review-toolkit.github.io/ort/docs/
Accessed: 2026-02.

[18] CHAOSS Community, *Metrics model: Starter project health*, [Online]. Available: https://chaoss.community/kb/metrics-model-starter-project-health/, Accessed: 2026-02.

[19] N. Zahan, P. Kanakiya, B. Hambleton, S. Shohanuzzaman, and L. Williams, *Openssf scorecard: On the path toward ecosystem-wide automated security metrics*, arXiv:2208.03412, 2022.

[20] S. P. Goggins, M. Germonprez, and K. Lumbard, "Making open source project health transparent," *Computer*, vol. 54, no. 8, pp. 104–111, 2021. DOI: 10.1109/MC.2021.3084015

[21] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021. DOI: 10.1007/978-1-0716-1418-1

[22] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324

[23] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451

[24] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432

[25] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd International Conference on Machine Learning*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430

[26] E. Krause, *Oss risk radar*, [Online]. Available: https://github.com/esuarkeN/oss-risk-radar/tree/680b7b4e747b254cc358b175722265b6d282a639, Source repository, pinned at commit 680b7b4. Accessed: 2026-06, 2026.

[27] E. Krause, *Historical dataset builder*, [Online]. Available: https://github.com/esuarkeN/oss-risk-radar/blob/680b7b4e747b254cc358b175722265b6d282a639/docs/methodology/historical-dataset-builder.md, Pinned at commit 680b7b4. Accessed: 2026-06, 2026.

[28] GH Archive, *Gh archive: Github archive data*, [Online]. Available: https://www.gharchive.org/, 2026.

[29] deps.dev, *Deps.dev api*, [Online]. Available: https://docs.deps.dev/api/v3/, Accessed: 2026-02.

[30] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.

[31] GeeksforGeeks. "Why is python the best-suited programming language for machine learning?" Accessed 2026-06-04. [Online]. Available: https://www.geeksforgeeks.org/why-is-python-the-best-suited-programming-language-for-machine-learning/

[32] Stack Overflow. "Technology | 2025 stack overflow developer survey." Accessed 2026-06-04. [Online]. Available: https://survey.stackoverflow.co/2025/technology

[33] GitHub. "Octoverse: The state of open source." Accessed 2026-06-04. [Online]. Available: https://octoverse.github.com/

[34] npm, Inc. “About npm.” Accessed 2026-06-04. [Online]. Available: https://docs.npmjs.com/about-npm

A Appendix

A.1 Observable Health Indicator Categories

Table A.1 summarizes the categories of observable health indicators considered in this thesis (Section 2.2): an example of the signals in each category, the public source they are derived from, and their expected relevance to inactivity risk. The categories motivate the feature groups implemented in Section 4.7; the exact per-feature definitions are given in Section A.2 below. Security-practice indicators are listed for completeness but, as discussed in Section 3.3, are kept as parallel triage context rather than as inputs to the inactivity model.

Table A.1: Categories of observable health indicators and their expected relevance

Category	**Example signals**	**Public source**	**Expected relevance**
Commit activity	Commits over recent windows; active commit months; days since last commit	GH Archive push events	Direct evidence of ongoing development
Contributor activity	Recent and new contributors; contributor concentration	GH Archive commit actors	Team breadth and single-maintainer (bus-factor) risk
Issue and PR activity	Opened/closed issues; PR merge ratio; response and merge latency	GH Archive issue/PR events	Maintenance interaction and responsiveness
Release activity	Releases; days since last release; versions published	GH Archive releases and package registry	Whether usable versions still reach users
Age and popularity	Repository and package age; stars; forks; popularity tier	GitHub metadata and package registry	Maturity and reach context
Security practice	Selected OpenSSF Scorecard checks	OpenSSF Scorecard	Parallel security-posture context, not an inactivity predictor

A.2 Full-History Feature Reference

The full-history model uses feature set `feature-set-v3-full-history`, which comprises 43 features grouped as summarized in Table A.2. Because the feature names are compact identifiers whose exact meaning is not obvious to a reader who is new to the artifact, this appendix documents each feature individually. The identifiers used here are the same strings that appear in the training code, in the exported model artifacts, and in the runtime interface, so the table doubles as a shared glossary across the thesis and the software.

Three conventions apply throughout and explain the recurring suffixes in the names. First, a numeric suffix denotes a trailing time window that ends at the observation time t: `_30d`, `_90d`, and `_365d` count events in the 30, 90, and 365 days immediately before t, and `_at_obs` denotes a state measured exactly at t. Second, all commit- and contributor-based counts use a human-actor filter, so activity generated by bots does not inflate the maintenance evidence. Third, wherever an absolute count would not be comparable across projects of very different sizes, the feature set also provides a relative or trend-based form (a ratio, a share, or a year-over-year change), because a large project and a small project can be equally well or badly maintained at very different absolute volumes. Every value is computed from information available at or before t only; the reasoning behind this restriction, and the distinction between it and the legitimate activity-persistence effect, is discussed in Section 4.9. The ecosystem one-hot flags and the repository-mapping and direct-dependency indicators are part of the runtime scoring vector rather than the reconstructed full-history set and are therefore not listed here.

Table A.3: Full-history feature reference (`feature-set-v3-full-history`): exact definition and rationale for each of the 43 features.

Feature	**Definition (all measured at or before t)**	**Why it is included**
Commit activity		
`commits_30d`	Number of human commits in the 30 days before t.	Short-term development pulse; captures whether the project is moving right now.
`commits_90d`	Number of human commits in the 90 days before t.	Quarter-scale activity level that is less noisy than the 30-day count.
`commits_365d`	Number of human commits in the 365 days before t.	Annual development volume; also the baseline against which the year-over-year drop is measured.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
active_commit_months_365d	Count of distinct calendar months (0–12) in the last year that contain at least one human commit.	Separates steady maintenance from a single burst: 300 commits in one month is a weaker health signal than the same work spread across ten months.
days_since_last_commit	Days between t and the most recent human commit.	Direct staleness signal; a long silence is one of the strongest indicators of impending inactivity.
Contributor activity and concentration		
contributors_90d	Distinct human contributors in the last 90 days.	Breadth of recent participation; a proxy for the size of the active team.
contributors_365d	Distinct human contributors in the last 365 days.	Annual contributor base; the reference for the contributor drop.
new_contributors_365d	Contributors active in the last year who never committed before the window started.	Measures inflow of fresh maintainers; a project with no newcomers is more exposed to attrition.
top1_contributor_commit_share_365d	Share (0–1) of the last year's commits made by the single most active contributor.	Key-person dependency: a high share means the project is fragile if that one person leaves.
top2_contributor_commit_share_365d	Combined commit share of the two most active contributors in the last year.	Detects a very small core team even when no single person dominates.
contributor_concentration_index	Herfindahl index: sum of squared per-contributor commit shares over the last year.	A single scalar for how concentrated contributions are (1.0 = one person, near 0 = evenly spread).
maintainer_concentration_flag	1 if the top contributor share ≥ 0.7 or the top-two share ≥ 0.85, else 0.	Binary bus-factor alarm for the clearly single-maintainer case.
Issue activity and backlog		
opened_issues_90d	Issues created in the last 90 days.	Inbound user demand and engagement with the project.
closed_issues_90d	Issues closed in the last 90 days.	Maintainer throughput on the issue tracker.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
`issue_closure_ratio_90d`	Closed divided by opened issues over the last 90 days.	Whether maintainers keep pace with inflow, expressed relatively so it is comparable across project sizes.
`issue_backlog_growth_90d`	Relative change in the open-issue count between $t - 90$ days and t.	A backlog that is trending upward signals that maintainers are losing control; size-normalized.
`stale_open_issues_count_at_obs`	Issues opened before $t - 90$ days that are still open at t.	Long-unaddressed items indicate neglect of the tracker.
Pull-request activity		
`opened_prs_90d`	Pull requests created in the last 90 days.	Contribution inflow from outside the core team.
`merged_prs_90d`	Pull requests merged in the last 90 days.	Direct evidence that maintainers are still integrating outside work.
`closed_unmerged_prs_90d`	Pull requests closed without merging in the last 90 days.	A high count relative to merges can signal rejection or disengagement from contributions.
`pr_merge_ratio_90d`	Merged divided by opened pull requests over the last 90 days.	Responsiveness to contributors, expressed relatively.
`stale_open_prs_count_at_obs`	Pull requests opened before $t - 90$ days that are still open at t.	Ignored contributions discourage further contribution and indicate maintainer absence.
Release activity		
`releases_365d`	GitHub releases published in the last 365 days.	Shipping cadence: whether usable versions still reach users.
`days_since_last_release`	Days since the most recent release or package version.	Release staleness; complements commit staleness for projects that ship rarely.
`versions_published_365d`	Package-registry versions published in the last 365 days.	Package-level shipping, which can differ from repository releases.
Age and popularity proxies		
`package_age_days`	Days from the first published package version to t.	Maturity context: young and long-established packages have different baseline inactivity risk.
`repo_age_days`	Days from repository creation to t.	Normalizes gaps and cadence against how long the project has existed.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
`stars_total_at_obs`	Cumulative stars observed up to t (a lower-bound proxy reconstructed from archive events).	Popularity and visibility; more attention tends to correlate with continued maintenance.
`forks_total_at_obs`	Cumulative forks observed up to t.	Reuse and derivative activity around the project.
`dependency_count_at_obs`	Declared dependency count of the resolved package version at t.	Proxy for maintenance burden and integration complexity.
`popularity_tier_at_obs`	Low / medium / high tier (0/1/2) derived from star and fork thresholds.	Coarse popularity bucket, because inactivity dynamics differ between obscure and widely used projects.
Diagnostic and binary flags		
`repo_archived_at_obs`	1 if the repository is archived by t, else 0.	Retained as diagnostic metadata only; already-archived rows are excluded from fitting so archival cannot become a shortcut label.
`has_recent_release_flag`	1 if any release or package version appeared in the last 365 days.	Simple "is it still shipping" indicator that is robust when exact counts are sparse.
`has_recent_pr_merge_flag`	1 if any pull request was merged in the last 90 days.	Simple "are maintainers still integrating" indicator.
Derived risk proxies		
`activity_drop_365d_vs_prev_365d`	Relative decline in commits from the previous year to the last year (positive = fewer commits).	Captures deceleration, enabling earlier detection of decline than the absolute level alone.
`contributors_drop_365d_vs_prev_365d`	Relative decline in the contributor count from the previous year to the last year.	Detects a shrinking maintainer base before commits fully stop.
`release_gap_risk`	Score in [0,1] for how overdue the next release is relative to the project's own release cadence (age-based when no release history exists).	Normalizes release silence against the project's normal rhythm rather than an absolute threshold.
`concentration_risk_score`	Composite score in [0,1] combining top-1 and top-2 shares, the Herfindahl index, and the concentration flag.	A single interpretable bus-factor risk value that consolidates the concentration features.
Responsiveness and backlog pressure		

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
`issue_first_response_median_days_365d`	Median days from issue creation to the first maintainer response over the last year.	Slowing first responses indicate disengaging maintainers, independently of resolution speed.
`issue_resolution_median_days_365d`	Median days from issue creation to closure over the last year.	How quickly reported problems are actually resolved.
`stale_issue_share_at_obs`	Stale open issues divided by the current open backlog.	The fraction of the backlog that is old, expressed relatively so it is comparable across sizes.
`pr_response_median_days_365d`	Median days to the first response, merge, or close on pull requests over the last year.	Responsiveness to contributors, the pull-request counterpart to issue first response.
`pr_merge_latency_median_days_365d`	Median days from creation to merge for merged pull requests over the last year.	Integration speed for work that is ultimately accepted.

A.3 Implementation Detail

This appendix collects the engineering detail of the artifact that is needed to reproduce or operate it, but that is not required to follow the scientific argument in Chapter 5. The cooperating system components referenced in Section 5.1 are summarized in Table A.4.

A.3.1 Technology Selection

The components use different programming languages, each selected for the part of the system where it is most appropriate.

Python is used for all data engineering and machine learning, including the historical dataset builder, the feature and label computation, the model implementations, and the scoring service. Python was chosen because it has the strongest ecosystem support for machine learning and data science work [31], [32], which makes it the natural environment for the analytical core of the artifact. The scoring service is implemented with FastAPI, and numerical work uses numpy and xgboost.

Go is used for the backend orchestration service. The backend is primarily a concurrent input/output workload: it calls several external providers and processes durable jobs. Go's static typing, its straightforward concurrency model, and its suitability for long-

Table A.2: Implemented full-history feature groups

Feature group	Example features
Dependency context	Dependency count of the resolved package version at the observation time.
Commit activity	Commits in the last 30, 90, and 365 days; active commit months; days since last commit.
Contributor activity	Contributors in the last 90 and 365 days; new contributors; contributor concentration.
Issue activity	Opened and closed issues; issue closure ratio; issue backlog growth; stale open issues.
Pull request activity	Opened, merged, and closed-unmerged pull requests; pull request merge ratio; stale open pull requests; pull-request response and merge latency.
Release activity	Releases in the last 365 days; days since last release; versions published in the last 365 days; recent release flag.
Age and popularity proxies	Package age; repository age; stars and forks observed in the available historical data; popularity tier.
Derived risk proxies	Activity drop; contributor drop; release gap risk; concentration risk score.
Responsiveness and backlog pressure	Issue first-response time; issue resolution time; stale issue share; pull-request response and merge latency.

Table A.4: Implemented system components

Component	Technology	Responsibility
Scoring and training workspace	Python	Historical dataset builder, feature engineering, model training, model artifacts, and the runtime scoring service.
Backend API	Go	Analysis orchestration, provider enrichment, durable job processing, and model-artifact scoring calls.
Analyst frontend	TypeScript / Next.js	Submission, repository overview, risk views, and machine-learning evaluation dashboards.
Shared schemas	TypeScript	Reusable data contracts shared by the frontend.
Orchestration scripts	Node.js	Seed generation, dataset building, notebook execution, and artifact staging commands.
Deployment assets	Docker / Kubernetes	Container images, Compose, and the Argo CD-managed Kubernetes deployment.

running network services make it a good fit for this role.

TypeScript with the Next.js framework is used for the analyst frontend, and TypeScript is also used for the shared data contracts. TypeScript provides type safety for the user interface and shares type definitions with the backend contracts. The frontend additionally relies on the broader JavaScript and Node.js tooling ecosystem, which is consistent with JavaScript being one of the most widely used languages on GitHub and the primary language of the npm ecosystem that the artifact analyzes [33], [34].

Node.js additionally hosts the orchestration command-line scripts under `scripts/ml`. These scripts coordinate the higher-level workflow steps, such as generating the repository seed, building the dataset, executing the training notebook, and staging artifacts. They wrap the underlying Python pipeline so that the full reproducible workflow can be triggered through consistent `npm` commands.

A.3.2 Dataset-Builder Modules

The historical dataset builder is divided into focused modules, summarized in Table A.5, so that ingestion, event parsing, feature computation, and label computation can be tested independently.

Table A.5: Dataset builder modules

Module	Responsibility
`adapters`	deps.dev, GitHub, and npm/PyPI registry adapters used for package-to-repository resolution and stable metadata.
`events`	GH Archive parsing into normalized repository histories of commits, issues, pull requests, releases, stars, and forks.
`features`	Observation-time feature computation from history before t.
`labels`	12-month maintenance and inactivity label computation from the future window.
`sampling`	Candidate sampling and popularity-tier derivation.
`pipeline`	The `DatasetBuilder` orchestrating ingestion, snapshots, labels, and export.
`storage`	JSON and JSON Lines reading and writing of the intermediate artifacts.
`cli`	The command-line entry point used by the orchestration scripts.

The `DatasetBuilder` class executes the construction workflow described in the methodology. The `ingest_candidates` step loads and de-duplicates seed candidates, resolves each package to a GitHub repository, and writes the `repositories.jsonl`, `packages.jsonl`, and `package_repository_links.jsonl` files. Candidates that can-

not be mapped to a GitHub repository, and forks when forks are excluded, are dropped at this stage. The `build_snapshots` step iterates over quarterly observation dates and, for each repository, constructs an `ObservationSnapshot` with explicit feature and label windows before computing the observation-time feature row. The `build_labels` step computes the future-window label for every snapshot, and `export_training_dataset` assembles the final training rows and the runtime repository feature cache.

A.3.3 Dataset-Quality Reporting

To make the per-run documentation items required by the methodology (Section 4.6) reproducible rather than transcribed by hand, a small `dataset_quality` module computes a structured quality report from the exported snapshot rows. It produces three views. The first is row accounting: the total, labeled, and unlabeled counts, the inactive and active label balance, the number of already-archived-at-observation rows that are excluded from fitting, and the completeness signals introduced above, including the `repo_silent_before_horizon` count that quantifies how many labeled examples the corrected coverage gate retains. The second is label balance per slice, broken down by ecosystem and by popularity tier, which exposes where labeled examples are concentrated. The third is per-feature missingness, derived from the missing-signal annotations already attached during feature extraction, so that a feature is reported as missing only when its underlying observation-time signal was unavailable rather than genuinely zero. The report is rendered in the training notebook alongside the existing coverage summary, and the row accounting deliberately records whether label-completeness signals are present in the export so that a value of zero is not misread as an absence of incomplete windows when an older export simply did not carry them.

A.3.4 Model Configurations

The Logistic Regression model is implemented directly in Python. It standardizes each input feature using training-set means and standard deviations, stores these standardization parameters with the model, and fits the coefficients by gradient descent on the class-weighted log-likelihood. Class-balanced sample weights counter the label imbalance, and an L2 penalty regularizes the coefficients. Because the model stores explicit per-feature coefficients, it doubles as an interpretable baseline whose weights can be inspected to check whether the engineered features behave as expected. This interpretability is what made it useful during the feature-set refinement described in Section A.3.12.

The neural network is a multilayer perceptron implemented on top of numpy, without a deep-learning framework. It uses two hidden layers of 64 and 32 units with ReLU activations and a sigmoid output, standardizes its inputs in the same way as the Logistic Regression model, and initializes its weights with Xavier-uniform values under a fixed random seed for reproducibility. Training uses mini-batch stochastic gradient descent on the binary cross-entropy objective, with balanced class weights and L2 regularization. The default configuration matches the methodology: a learning rate of 0.01, 150 epochs, a batch size of 256, and an L2 penalty of 0.001. The forward and backward passes are written explicitly, which keeps the model transparent and removes any heavy training dependency from the artifact.

The XGBoost model wraps the `xgboost` library. It trains a gradient-boosted tree ensemble with a binary-logistic objective and the same class-balanced sample weights as the other models. The default configuration uses 80 boosting rounds, a maximum tree depth of three, a learning rate of 0.08, row and column subsampling of 0.9, L2 leaf regularization, the histogram tree method, and a fixed seed. After training, the trained booster is serialized to its JSON representation and stored inside the model artifact, and the gain-based feature importances are computed and normalized so they can be displayed in the evaluation dashboard.

A.3.5 Artifact Serialization

Each trained model is serialized to a self-describing JSON artifact. The artifact stores the model family and version, the ordered feature names, the model parameters (coefficients and standardization for Logistic Regression, weights and biases for the neural network, or the serialized booster for XGBoost), the calibration bins, the decision threshold, the feature-set version, and the training timestamp. Because the feature names and standardization are part of the artifact, the runtime service can construct the feature vector in the correct order and apply the same transformations without sharing code paths with the training process. Each training run is written to a run file under the training runs directory together with the dataset hash and the held-out metrics, and a `latest-run.json` pointer records the most recently produced run.

A.3.6 Training Orchestration

The headless command `npm run ml:train` executes the training notebook non-interactively and writes the run artifacts and the latest-run pointer, so that the same notebook is used for both interactive exploration and reproducible artifact generation. The end-to-end

command `npm run ml:bootstrap` chains dataset construction and notebook-based training, and the foundation variants of these commands use the preserved 5,000-repository seed and the filtered GH Archive cache for the thesis-grade run. The orchestration scripts wrap the Python pipeline rather than reimplementing it; they pass through the seed file, the GH Archive source, the output directories, the observation window, and the dataset-size guardrails, which keeps the reproducibility parameters explicit at the command line.

Listing A.1: Headless notebook-based training

```
npm run ml:train
```

A.3.7 Artifact Staging and Promotion Gate

Trained artifacts are not deployed automatically. The staging command `npm run ml:stage-training` validates a candidate training bundle and promotes it into the deployed bundle under `deployment/training` only if it passes a set of quality gates. This gate is the safety boundary between experimentation and deployment.

The staging script first validates the dataset itself: the snapshots must be non-empty, every labeled snapshot must come from a real GitHub repository, and the bundle must contain at least a configured minimum number of distinct repositories and distinct inactive repositories. It then validates the model run artifacts: all required model artifacts must be present and complete for the latest dataset hash, and their feature versions must match the expected full-history and cold-start feature sets. This prevents a partial or inconsistent bundle, for example one missing the cold-start models, from reaching deployment.

The most important gate is the regression comparison. Before promotion, the candidate artifacts are compared against the currently deployed baseline. For each required model, the script checks that ROC-AUC has not dropped, and that the Brier score has not increased, by more than a configured tolerance. If any required model fails this comparison, promotion is rejected and the existing staged bundle is retained. When promotion succeeds, the script normalizes the snapshot file, the run artifacts, and the repository feature cache to their runtime paths and writes them into the deployed bundle.

A.3.8 Scoring Service

The scoring service is a FastAPI application under `mltraining/scoring/app`. It exposes a small internal interface: health and readiness probes, a `/features/extract` endpoint for feature extraction, and the `/score/model` endpoint that scores a batch of dependencies with a supplied model artifact. The service does not own a model registry; instead, the backend passes the staged artifact in the scoring request, which keeps the scoring service stateless and makes the deployed model an explicit input rather than hidden state.

For each dependency, the service builds the feature vector in the artifact's feature order, imputing missing signals as neutral zero values and recording which signals were missing. It deserializes the artifact according to its model family, computes the raw probability, and applies the stored calibrator to obtain the calibrated inactivity probability. This probability is scaled to a 0–100 inactivity-risk score, and a complementary 12-month maintenance-outlook score is derived as its inverse. The service assigns a discrete risk bucket and an action level from the score, where the action level is one of `monitor`, `review`, or `replace_candidate`, so that the continuous score maps onto the triage actions used in the demonstrator.

The service keeps the inactivity prediction and the security posture strictly separate, exactly as required by the methodology. The OpenSSF Scorecard signal is summarized into a parallel security-posture score and is never fed into the predictive feature vector [19]. A confidence value is computed from the completeness of the available input signals and the decisiveness of the prediction, and explicit caveats are attached when calibration bins are missing, when the artifact's feature version is unexpected, or when input signals had to be imputed. Finally, the service returns human-readable explanation factors and evidence items so that an analyst can see which signals shaped the score rather than only the number itself.

A.3.9 Backend Orchestration Service

The backend is a Go service under `backend/api`. It exposes the public REST API for analyses, repositories, jobs, and the training dataset and run endpoints. Its central responsibility is to turn a submission into an enriched, scored analysis reliably, even in the presence of slow or failing external providers.

Analyses are processed through a durable job queue backed by PostgreSQL. When a submission arrives, the service creates an analysis record and a job record in a single

transaction and returns immediately. A background worker leases the next pending job, processes it, and persists the result. If processing fails, the job is retried with a bounded number of attempts and a retry delay, and it is only marked as failed once the attempts are exhausted. This design makes the analysis lifecycle resilient and observable, and it decouples the user-facing request from the slower enrichment and scoring work. For repository submissions, the service also reuses a recent completed analysis when one exists for the same repository and the same staged model set, which avoids redundant re-enrichment.

During processing, the worker enriches each repository and scores it. Enrichment fetches stable repository metadata from GitHub and retrieves the OpenSSF Scorecard signal, attaching the raw signals to the repository for display. Each enrichment provider is implemented as a separate adapter, so a single failing provider degrades the available signals rather than the whole analysis.

The backend implements the two prediction regimes from the methodology at runtime. When it enriches a repository, it attempts to look the repository up in the staged repository feature cache. On a cache hit, the repository is scored in the full-history regime using the cached observation-time features; on a miss, it falls back to the cold-start regime using only approximate features derived from live metadata, and a caveat records this. When several model artifacts are staged for the same dataset hash and regime, the backend scores the repository with each and combines the results into an ensemble profile by averaging the scores, while also retaining the per-model outputs and adding a model-agreement explanation factor that reports the spread between models.

A.3.10 Analyst Frontend

The frontend is a Next.js application under `frontend/web` that uses the React component model and the application router. It provides the analyst-facing workflow and the evaluation dashboards used during the thesis. Submission is handled through a form that accepts one or more repository URLs or a demo analysis. Once an analysis completes, the dashboard presents the repository overview with risk distribution and filtering, and a dedicated view renders the per-repository detail card with its risk profile, caveats, and evidence.

A second area of the frontend is the machine-learning evaluation surface. These pages read the training dataset and run endpoints exposed by the backend and visualize them with purpose-built charts, including a calibration curve, a model-metric comparison across the trained models, a Logistic Regression coefficient view, and a training-metric

history. These views were used directly to inspect and report model behavior during the thesis.

A.3.11 Deployment

The artifact is containerized for both local and production-style operation. Canonical image definitions live under `deployment/docker`, and a Docker Compose file orchestrates PostgreSQL, the Go backend, the scoring service, and the frontend for local development; a single `npm run dev` command builds and starts the full stack and waits for the services to become healthy. For production-style operation, the project is deployed to a k3s cluster managed by Argo CD, with container images published to a registry and the application served behind a public domain [26]. Because the deployed model bundle is a versioned set of files under `deployment/training`, deployment and model promotion remain auditable: the running service uses exactly the artifacts that passed the staging gate described in Section A.3.7.

A.3.12 Feature-Set Refinement

The feature set used for training was refined iteratively, as described in the methodology's formative development step. The first feature set contained variables that turned out to be poor point-in-time predictors. A representative example is a raw open-issue count: large, actively maintained projects can carry very large issue backlogs, so a model can wrongly associate a high absolute issue count with risk, even though the project is healthy. The revised feature set therefore replaces raw counts with relational and trend-based variables, such as issue closure ratio, issue backlog growth, stale-issue share, and pull-request response and merge latency, which are more comparable across projects of different sizes.

A second class of problems came from repository identity. Some packages point to mirror repositories rather than to the upstream development repository, which can make a project look inactive even though active development happens elsewhere. The mapping logic therefore prefers explicit seed and deps.dev mappings and resolves the primary package-to-repository link per repository, rather than treating every registry entry as an independent signal. Finally, the revised feature set keeps `repo_archived_at_obs` only as diagnostic metadata and excludes already-archived observation rows during model fitting, so that the visible archival flag cannot become a shortcut label.

A.3.13 Slice Evaluation

A single aggregate metric can hide a subgroup on which the model performs poorly, because a strong majority subgroup can mask a weak minority one. A slice-evaluation helper therefore re-scores the same held-out predictions within subgroups, reusing the existing metric implementation so that no separate evaluation logic is introduced. To make this possible the training pipeline also returns the per-row held-out predictions alongside the aggregate result; these are used only for analysis and are not written into the model artifact, so the serialized artifacts and the reported headline metrics are unchanged. The harness includes a consistency check that requires the aggregate over all slices to reproduce the model's reported held-out ROC-AUC, so the per-slice figures are guaranteed to originate from the same execution as the headline metrics.

The subgroups are those that vary meaningfully in the foundation dataset: popularity tier, repository-age band, a cold-start-like versus has-history regime, and whether the repository was still active at the observation date. The last of these is the active-at-t subset referenced in the evaluation strategy (Section 4.11) and in the discussion of activity persistence in Chapter 8: restricting the evaluation to repositories that are still active at t separates genuine early-warning skill from the trivial tendency of already-quiet repositories to remain quiet. Ecosystem is deliberately excluded as a slice, because the repository-first foundation seed assigns the same provenance ecosystem to every row and therefore holds it effectively constant; cross-ecosystem evaluation would require a package-first sampling frame and is left to future work. Because slicing reduces the sample size, each slice is reported with its own row count, ranking metrics that are undefined for single-class slices are reported as absent rather than as a misleading value, and thin slices are interpreted with corresponding caution.

A.3.14 Repository-Disjoint Robustness Split

The reported evaluation uses a chronological split: rows are ordered by observation date and cut into training, validation, and test partitions. Because observations are taken quarterly, a repository contributes several snapshots over time, so the same repository can appear in both the training and the test partition, and the twelve-month label windows of its successive snapshots overlap. The held-out metric then partly reflects generalization to later observations of repositories already seen during training, which is optimistic relative to performance on genuinely new repositories. This is the deployment-realistic question — the system does re-score repositories over time — but it should not be the only question asked.

A further evaluation iteration therefore adds a repository-disjoint split that assigns every snapshot of a repository to a single partition, so that the held-out set contains only repositories absent from training. The split is selectable through a configuration option whose default preserves the chronological behavior, so the reported artifacts and headline metrics are unchanged, and the pipeline asserts that the partitions are repository-disjoint whenever the grouped split is used. Running the deployed full-history model under both splits lets the difference between them bound the optimism introduced by repository recurrence. In practice the held-out ROC-AUC decreases only marginally under the repository-disjoint split, which indicates that the model's discrimination is not an artifact of memorizing individual repositories but generalizes to unseen ones; the exact figures for both splits are produced in the training notebook and reported in Chapter 6.

A.4 Dataset Construction Steps

The historical dataset builder creates its dataset through the sequence of intermediate steps summarized in Table A.6, writing the intermediate files listed below under the configured output directory so that the construction process is auditable and the generated dataset can be inspected before training.

Table A.6: Historical dataset construction workflow

Step	Description
1	Load repository candidates from the foundation seed.
2	Use the seed strata to obtain a mixture of active, dormant, and archived repository histories.
3	Resolve repository identities from explicit seed fields or package metadata where required.
4	Enrich stable repository metadata through GitHub.
5	Parse GH Archive event files into normalized repository histories.
6	Build quarterly observation snapshots for each repository.
7	Compute observation-time features using only data available at or before t.
8	Compute 12-month labels using only the future window $(t, t +$ 12 months].
9	Export the final snapshot dataset for the existing model-training workflow.
10	Write a repository feature cache for runtime full-history scoring.

The documented intermediate outputs are `repositories.jsonl`, `packages.jsonl`, `package_repository_links.jsonl`, `observation_snapshots.jsonl`, `snapshot_features.jsonl`, `snapshot_labels.jsonl`, `training_snapshots.json`, and `repository-feature-cache.json`.

A.5 Evaluation Metric Definitions

All metrics are computed on the calibrated probabilities of the held-out test slice. Let N be the number of evaluation samples, $p_i \in [0,1]$ the predicted probability of inactivity for sample i, and $y_i \in \{0,1\}$ the true label (1 = inactive within twelve months). The positive (inactive) base rate of the slice is

$$\bar{p} = \frac{1}{N}\sum_{i=1}^{N} y_i, \tag{A.1}$$

reported as the *inactive 12m rate*.

Ranking. ROC-AUC is computed as the Mann–Whitney statistic over all positive–negative pairs, that is, the probability that a random inactive repository is scored above a random active one, with ties counted as one half:

$$\mathrm{AUROC} = \frac{1}{N_+ N_-}\sum_{i:\,y_i=1}\sum_{j:\,y_j=0}\Big(\mathbb{1}[p_i > p_j] + \tfrac{1}{2}\mathbb{1}[p_i = p_j]\Big), \tag{A.2}$$

where N_+ and N_- are the numbers of positive and negative samples. It is threshold- and calibration-independent: 0.5 is random ranking and 1.0 is a perfect ranking.

Probabilistic accuracy. The Brier score is the mean squared error between probability and outcome, and the log loss is the binary cross-entropy (with a small ε for numerical stability):

$$\mathrm{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2, \tag{A.3}$$

$$\mathrm{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\Big[\, y_i \ln p_i + (1-y_i)\ln(1-p_i)\,\Big]. \tag{A.4}$$

Both are lower-is-better and penalize miscalibration; the log loss grows without bound as a confident prediction approaches the wrong outcome, so it punishes overconfidence more sharply than the bounded Brier score.

Calibration. The expected calibration error partitions $[0,1]$ into $B = 10$ equal-width bins. For bin b with member set $\mathcal{B}_b$, let conf_b be the mean predicted probability and acc_b the empirical positive rate; the ECE is the sample-weighted average gap between

predicted confidence and observed frequency:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \Big| \,\mathrm{conf}_b - \mathrm{acc}_b \,\Big|. \tag{A.5}$$

Thresholded classification. A decision threshold τ maps probabilities to decisions $\hat{d}_i = \mathbb{1}[p_i \geq \tau]$, yielding the counts of true and false positives and negatives (TP, FP, FN, TN). From these,

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \qquad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \qquad \mathrm{F1} = \frac{2\,\mathrm{Precision}\cdot\mathrm{Recall}}{\mathrm{Precision}+\mathrm{Recall}}. \tag{A.6}$$

The F1 score is the harmonic mean of precision and recall, so it cannot be made large by optimizing one at the expense of the other; τ is selected on the validation slice to maximize it.

Combined quality score. The quality score converts ranking and probabilistic accuracy into skill scores relative to the trivial base-rate predictor and averages them, weighting ranking at 0.6 and calibration at 0.4. With the climatological Brier score $\bar{p}(1-\bar{p})$ and $\mathrm{clip}_{[0,1]}$ clamping to the unit interval,

$$s_{\mathrm{AUC}} = \mathrm{clip}_{[0,1]}\left(\frac{\mathrm{AUROC}-0.5}{0.5}\right), \tag{A.7}$$

$$s_{\mathrm{Brier}} = \mathrm{clip}_{[0,1]}\left(1-\frac{\mathrm{Brier}}{\bar{p}(1-\bar{p})}\right), \tag{A.8}$$

$$\mathrm{Quality} = \mathrm{clip}_{[0,1]}\Big(0.6\,s_{\mathrm{AUC}} + 0.4\,s_{\mathrm{Brier}}\Big). \tag{A.9}$$

A value of 0 indicates no improvement over predicting the base rate and 1 indicates perfect ranking and calibration. Because both skill terms are measured against the slice base rate $\bar{p}$, the quality score is conditional on the sampled prevalence and is intended for model comparison rather than as a standalone performance guarantee.

A.6 Reproducibility

The dataset builder and training pipeline are executed through repository scripts. The foundation seed can be generated with:

```
npm run ml:seed:foundation -- ‘
  --target-repositories 5000 ‘
```

```
  --github-token <TOKEN>
```

The larger foundation dataset can then be built and trained with:

```
npm run ml:bootstrap:foundation -- ‘
  --github-token <TOKEN> ‘
  --gharchive-source <GHARCHIVE_PATH>
```

The underlying full command path uses a generated foundation seed, a GH Archive source directory, a dedicated training output directory, a training snapshot export path, and a repository feature cache output path. The repository documentation recommends the foundation path for the thesis/training base, while the smaller default bootstrap path remains a local smoke-test workflow [27].

For each final training run, the following items should be documented:

- seed generation command and seed metadata,
- GH Archive coverage period,
- output directory,
- training snapshot path,
- repository feature cache path,
- dataset hash,
- model artifact path,
- model name and version,
- feature version,
- number of total, labeled, and unlabeled rows,
- train, validation, and test split sizes,
- held-out evaluation metrics.

The reported full-history run uses dataset hash:

```
c14ea4562c6e319d475c3387e68cf180879d403bff91d33c06b7fbc7430dbaab
```

and exports the model artifact:

`/app/tmp/training/runs/20260617T141157053388Z-xgboost-full-history-c14ea4562c6e.jso`

The corresponding source revision is archived at the commit referenced in the artifact citation [26], so that the reported run can be traced to a fixed state of the dataset builder and training pipeline.

A.7 Full Demonstrator Ranking

Table A.7 lists the complete ranked inactivity-risk view for all 30 dependency repositories of `ArchipelagoMW/Archipelago` produced by the deployed model in the demonstrator (Chapter 7); the body reports the flagged and medium-risk subset in Table 7.1.

A.8 Supplementary Evaluation Tables

This section collects the detailed evaluation tables whose findings are reported and interpreted in Chapter 6. Table A.8 gives the full per-subgroup slice evaluation of the deployed `xgboost-full-history` model (Section 6.3.3), and Table A.9 gives the cross-test of the human-actor filter (Section 6.4).

Table A.7: Complete ranked inactivity-risk view for the dependency repositories of `ArchipelagoMW/Archipelago`, as produced by the deployed model. Risk is the 0–100 inactivity-risk score; "Conf." is the per-prediction confidence; "Regime" is full-history (FH) or cold-start (CS).

Package	Source repository	Risk	Action	Conf.	Regime
bsdiff4	`ilanschnell/bsdiff4`	77.9	review	0.69	CS
cymem	`explosion/cymem`	46.6	monitor	0.81	CS
kivymd	`kivymd/KivyMD`	46.6	monitor	0.71	CS
pymem	`srounet/Pymem`	46.6	monitor	0.71	CS
markupsafe	`pallets/markupsafe`	46.6	monitor	0.81	CS
setproctitle	`dvarrazzo/py-setproctitle`	46.6	monitor	0.67	CS
jinja2	`pallets/jinja`	26.0	monitor	0.70	CS
schema	`keleshev/schema`	8.8	monitor	0.86	CS
platformdirs	`tox-dev/platformdirs`	8.8	monitor	0.91	CS
typing-extensions	`python/typing_extensions`	8.8	monitor	0.86	CS
pyshortcuts	`newville/pyshortcuts`	8.8	monitor	0.86	CS
pony	`ponyorm/pony`	8.8	monitor	0.86	CS
flask-caching	`pallets-eco/flask-caching`	8.8	monitor	0.91	CS
colorama	`tartley/colorama`	5.8	monitor	0.83	CS
kivy	`kivy/kivy`	5.8	monitor	0.94	CS
certifi	`certifi/python-certifi`	5.8	monitor	0.78	CS
orjson	`ijl/orjson`	5.8	monitor	0.89	CS
pathspec	`cpburnz/python-pathspec`	5.8	monitor	0.89	CS
werkzeug	`pallets/werkzeug`	5.8	monitor	0.89	CS
waitress	`Pylons/waitress`	5.8	monitor	0.94	CS
flask-compress	`colour-science/flask-compress`	5.8	monitor	0.78	CS
flask-limiter	`alisaifee/flask-limiter`	5.8	monitor	0.89	CS
flask-cors	`corydolphin/flask-cors`	5.8	monitor	0.89	CS
websockets	`python-websockets/websockets`	1.8	monitor	0.98	CS
pyyaml	`yaml/pyyaml`	1.8	monitor	0.93	CS
cython	`cython/cython`	1.8	monitor	0.98	CS
bokeh	`bokeh/bokeh`	1.8	monitor	0.87	CS
mistune	`lepture/mistune`	1.8	monitor	0.93	CS
psutil	`giampaolo/psutil`	1.8	monitor	0.81	CS
flask	`pallets/flask`	1.7	monitor	0.97	FH

Table A.8: Held-out ROC-AUC and Brier score of `xgboost-full-history` within subgroups of the test slice. "Inactive rate" is the positive rate in the subgroup.

Dimension	Subgroup	n	Inactive rate	ROC-AUC
Popularity	high	2178	0.48	0.968
Popularity	medium	1398	0.66	0.945
Popularity	low	607	0.64	0.689
Activity at t	active@t	1786	0.17	0.883
Activity at t	quiet@t	2397	0.85	0.860
History regime	has-history	2320	0.30	0.917
History regime	cold-start-like	1863	0.88	0.872

Table A.9: Cross-test of the human-actor filter: training on bot-contaminated features versus clean features, both evaluated against the identical clean (human-filtered) label on the held-out slice. A lower ROC-AUC or higher Brier for the contaminated features means the filter helps.

Model	ROC-AUC (clean)	ROC-AUC (contam.)	ΔROC-AUC
`logistic-regression-full-history`	0.9379	0.9373	−0.0007
`xgboost-full-history`	0.9529	0.9522	−0.0007
`logistic-regression-cold-start`	0.8889	0.8871	−0.0018
`xgboost-cold-start`	0.9488	0.9477	−0.0011